

Lectures examples

Lecture 1

1) Data Type = domain + interface VS ADT VS DS

DT	ADT	DS
• what values & what operations are allowed - int - float - char - bool... + - * / } op. ↑ = VALUES + Primitive operations	• what operations, not how they are implemented - stack - Queue - List - Map - Set = REPRESENTATION - Bag ex: stack ADT: push pop top isEmpty front	• how is the ADT represented in memory ex: for stack ADT: → dynamic array → SL → DLLA = Implementation

ex: ADT Queue

push
pop
front
isEmpty
top

interface (repres.)

Array
Circular Array
LL
Two stacks

DS

int
char
string

DT (inside)

ex: Queue of integers^u

int → DT.

Queue → ADT

Circular array → DS.

2) Pseudocode rules

@ ← (#TODO)

↑ ← pointer

// ← comm.

read ← read data

print ← print data

← ← assignment op.

= ← test equality

Indexing from 1!

3) subalgorithms vs functions

↓
don't return anything

↓
return sth.

4) pointers

$p: \uparrow \text{Integer} \Rightarrow p$ is a variable whose value is an address
 $\Rightarrow$ if we go at that address, we will find an Integer value stored there.

allocate(p)

free(p)

NIL (NULL)

5) generic types

$\text{TElem} \rightarrow$ elem of a generic type $\rightarrow$ Bag, Set, List, Queue, Stack

$\text{TComp} \rightarrow$ comparable elem $\rightarrow$ Sorted Bag, Heap

$\{ \text{TKey} \rightarrow$ part of a map $\rightarrow$ Map, Sorted Map, Multimap, Sorted Multimap

$\{ \text{Tvalue} \rightarrow$ value associated to the key

$\text{TElem} = \text{pair} \langle \text{TKey}, \text{Tvalue} \rangle$

Relation

Tfunction

6) Complexities

$T(m) =$ nr. of STEPS, depends on SIZE of INPUT.

ex: $T(m) = 2m^2 + 4m + 2 \Rightarrow m^2 =$ leading term.

COMPLEXITY = How does $T(m)$ change if m increases

O notation = UPPER BOUND.

ex: $T(m) = m^2 + 7m + 2 \Rightarrow m^2 =$ leading term.

anything $\geq m^2$ is a VALID UPPER BOUND.

Ω notation = LOWER BOUND.

anything $\leq m^2$ is a VALID LOWER BOUND $\uparrow$

Θ notation = ACTUAL, EXACT VALUE

only $= m^2$ is a valid value.

JBC, WC, AC only for those algorithms where the length/size of container DOES NOT MATTER.

ex: • sum of all even no in array

→ traverse if all $\Rightarrow$ NO $\begin{matrix} \uparrow \\ BC, WC, AC \end{matrix}$; length MATTERS

- first occurrence of a nr in an array

↳ length does not matter $\Rightarrow \exists Bc, Wc, Ac \Rightarrow \emptyset$.

Bc: no. of steps is $\min(T(n) = \min)$ } measured in Θ .

we: no. of steps is $\max (T(n)) = \max$

Ac: average

$$\int \varphi(I) \cdot E(I)$$

probability
of having I
as input

↓
how many times
we loop
(nr. of operations)

If $B_c = W_c = A_c \Rightarrow$ total complexity $T_c = \Theta(\dots)$

$$Bc \neq Wc \neq Ac \Rightarrow \tau_c = \underline{Wc} = O(\dots)$$

Ex: Compute complexity:

$$T(n) = \begin{cases} 1, & n \leq 1 \\ T(\frac{n}{2}) + 1, & \text{oth.} \end{cases}$$

→ Suppose $m = 2^k$:

$$T(2^k) = T\left(\frac{2^k}{2}\right) + 1 = T(2^{k-1}) + 1$$

$$T(2^k - 1) = T(2^{k-2}) + 1$$

•

$$T(2^1) = T(2^0) + 1$$

$$T(2^k) + T(2^{k-1}) + \dots + T(2^1) = T(2^{k-1}) + T(2^{k-2}) + \dots + T(2^1) + T(2^0) + \underbrace{+1 + \dots + 1}_k$$

$$\Rightarrow T(2^K) = T(2^0) + K = 1 + K.$$

$$T(1) = 1$$

$$m = 2^k \Rightarrow k = \log_2 m$$

$$\Rightarrow T(m) = 1 + \log_2 m \in \Theta(\log_2 m)$$

Ex: Solve the recurrence: $T(n) = \begin{cases} 1, & n \leq 1 \\ 1 + 3T(n/3), & \text{otherwise} \end{cases}$

→ Suppose $n = 3^k$

$$T(3^k) = 1 + 3T\left(\frac{3^k}{3}\right) = 1 + 3T(3^{k-1})$$

$$3T(3^{k-1}) = 3(1 + 3T(3^{k-2}))$$

⋮

$$3T(3^1) = 3(1 + 3T(3^0))$$

$$T(3^k) + \cancel{3T(3^{k-1})} + 3T(3^{k-2}) + \dots + 3T(3^1) = 1 + \cancel{3T(3^{k-1})} + 3 + 3^2T(3^{k-2}) + \dots + 3 + 3^2T(3^0)$$

$$\Rightarrow T(3^k) = 1 + 3T(3^{k-1})$$

$$= 1 + 3(1 + 3T(3^{k-2}))$$

$$= 1 + 3(1 + 3(1 + 3T(3^{k-3}))) \dots$$

$$= \dots$$

$$= 3^k + 3 \dots$$

Lecture 2

1) Arrays

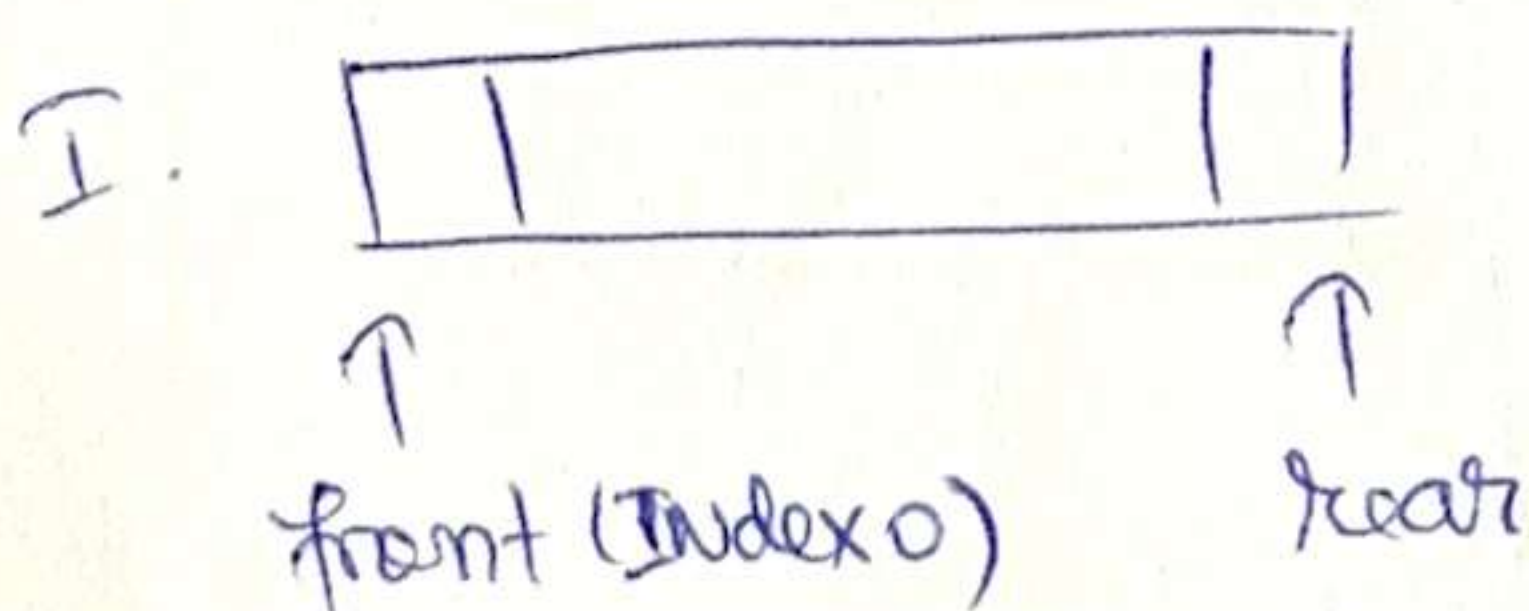
- DS, but sometimes treated like ADT.
- + / -
- Dyn. Arrays ...

Lecture 3 (see ref. sheet)

- Bag → Dyn. Array
(ADT) (DS)

Lecture 4

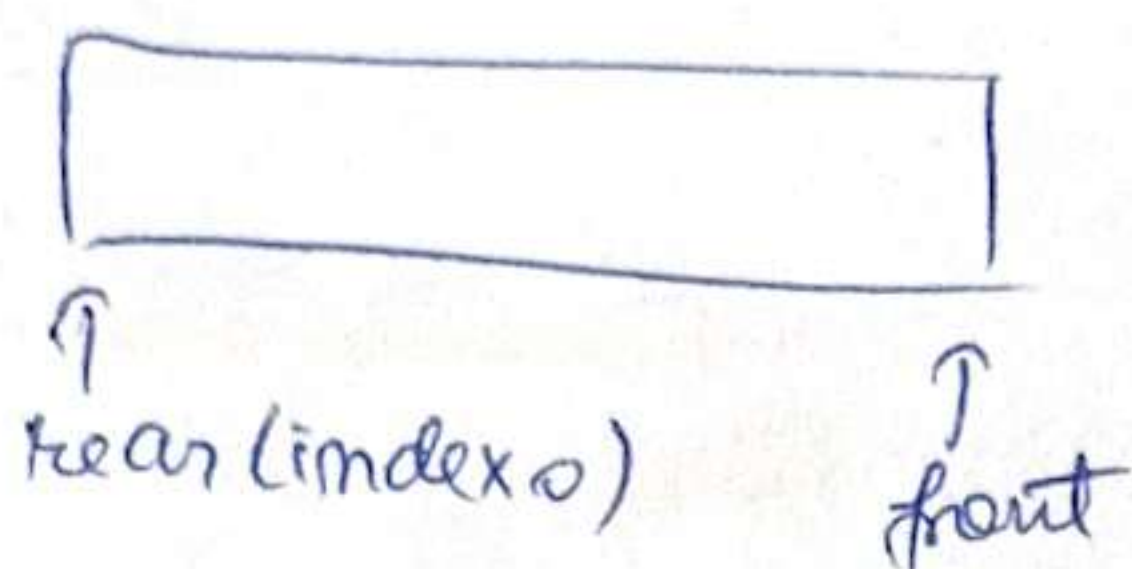
1) where to place FRONT / REAR in a Dyn. Array? (for a Queue).



push() ⇒ to rear. ⇒ $\Theta(1)$

pop() ⇒ from front ⇒ $\Theta(n)$ because we need to shift the rest of the

⇒ (GOOD IF WE PUSH MORE) elements so that we fill in the gap and INDEX=0.



pop() $\rightarrow$ from front $\Rightarrow \Theta(1)$

push() $\rightarrow$ to rear $\Rightarrow \Theta(n)$ (make space for it by shifting)

$\Rightarrow$ GOOD IF WE POP MORE

2) SL search

function search(sl, elem) is:

current $\leftarrow$ sl.head

while current $\neq$ Nil and [current].info $\neq$ elem ex position

 current $\leftarrow$ [current].next

search $\leftarrow$ current

"returns".

$\rightarrow$ return a boolean flag when you implement a container

BC: $\Theta(1)$ first node $\Rightarrow \Theta(n)$.

WC: $\Theta(n)$ last node

3) SL insert at the beginning

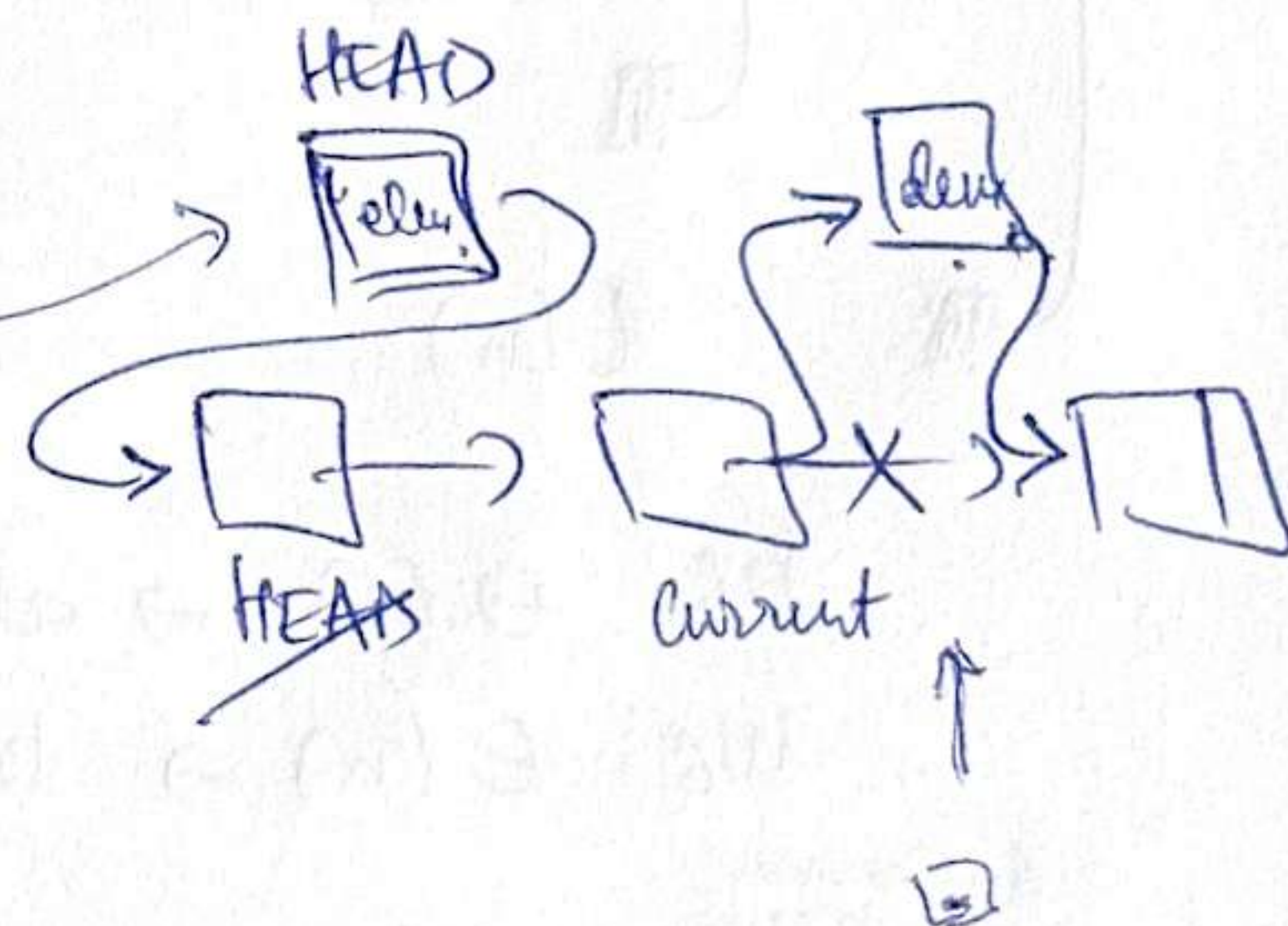
subalgorithm insertFirst(sl, elem) is:

newNode $\leftarrow$ allocate()

[newNode].info $\leftarrow$ elem

[newNode].next $\leftarrow$ sl.head

sl.head $\leftarrow$ newNode



4) SL insert after a node

subalg insertAfter(sl, currentNode, elem) is:

newNode $\leftarrow$ allocate()

[newNode].info $\leftarrow$ elem

[newNode].next $\leftarrow$ [currentNode].next

[currentNode].next $\leftarrow$ newNode

$\Theta(1)$

5) SL insert AT a given position

= traverse with currentNode the SL until pos found $\Rightarrow$

$\Rightarrow$ insert after currentNode

ex: at 5th position $\rightarrow$ stop at pos 4 $\Rightarrow$ insert after 4.

! subalg insertPosition (sll, pos, elem) is:

if pos < 1 then
| @error

else if pos = 1 then
| // insert beginning

else // traverse to find pos

currentNode ← sll.head

currentPos ← 1

while currentPos < pos - 1 and currentNode ≠ Nil ex

| currentNode ← [currentNode].next

| currentPos ← currentPos + 1

□

if currentNode ≠ Nil then

| // insert after.

else

| @error

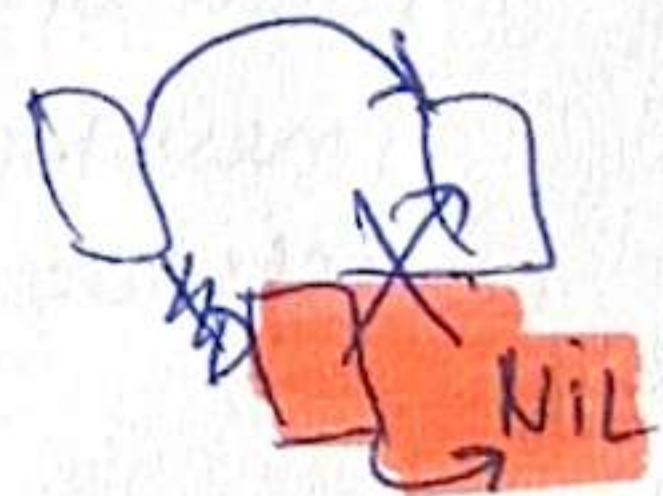
□

□

$O(n)$

BC: $\Theta(1) \rightarrow$ at front $\} \Rightarrow O(n)$

WC: $\Theta(n) \rightarrow$ traverse



c) delete given elem. (find the node BEFORE)

function deleteElement (sll, elem) is:

currentNode ← sll.head

prevNode ← Nil

while currentNode ≠ Nil and [currentNode].info ≠ elem ex

| prevNode ← currentNode

| currentNode ← [currentNode].next

□

if currentNode ≠ Nil and prevNode = Nil then // delete head.
| sll.head ← [sll.head].next (skip it)

else if currentNode ≠ Nil then // normal pos (skip current)

| [prevNode].next ← [currentNode].next

| [currentNode].next ← Nil

□

deleteElement ← currentNode

$O(n)$

Lecture 5

1) Iterator for SL.

subalg **init(it, sll)** is:

```
it.sll ← sll  
it.currentElement ← sll.head
```

$\Theta(1)$

function **getCurrent(it)** is:

```
if it.currentElement = Nil then
```

```
  @throw
```

```
e ← [it.currentElement].info
```

```
getCurrent ← e
```

$\Theta(1)$

subalg **next(it)** is:

```
if it.cur...
```

```
  @throw...
```

```
it.currentElement ← [it.currentElement].next
```

function **valid(it)** is

```
if it.currentEl ≠ Nil then
```

```
  valid ← True
```

```
else valid ← False
```

2) DLL.

subalg **insertLast(dll, elem)** is:

```
newNode ← allocate()
```

```
[newNode].info ← elem
```

```
[newNode].next ← Nil
```

```
[newNode].prev ← dll.tail
```

```
if dll.head = Nil then // empty list
```

```
  dll.head ← newNode
```

```
  dll.tail ← newNode
```

```
else
```

```
  [dll.tail].next ← newNode
```

```
  dll.tail ← newNode
```

$\Theta(1)$



Subalg insertPosition (dll, pos, elem) is:

$O(n)$

if pos < 1 then

| @error

else if pos = 1 then

| // insert First

else

currentNode ← dll.head

currentPos ← 1

while currentNode ≠ Nil and currentPos < pos - 1 ex

| currentNode ← [currentNode].next

| currentPos ← currentPos + 1

if currentNode = Nil

| @error

else if currentNode = dll.tail

| // insert Last

else

newNode ← allocate ()

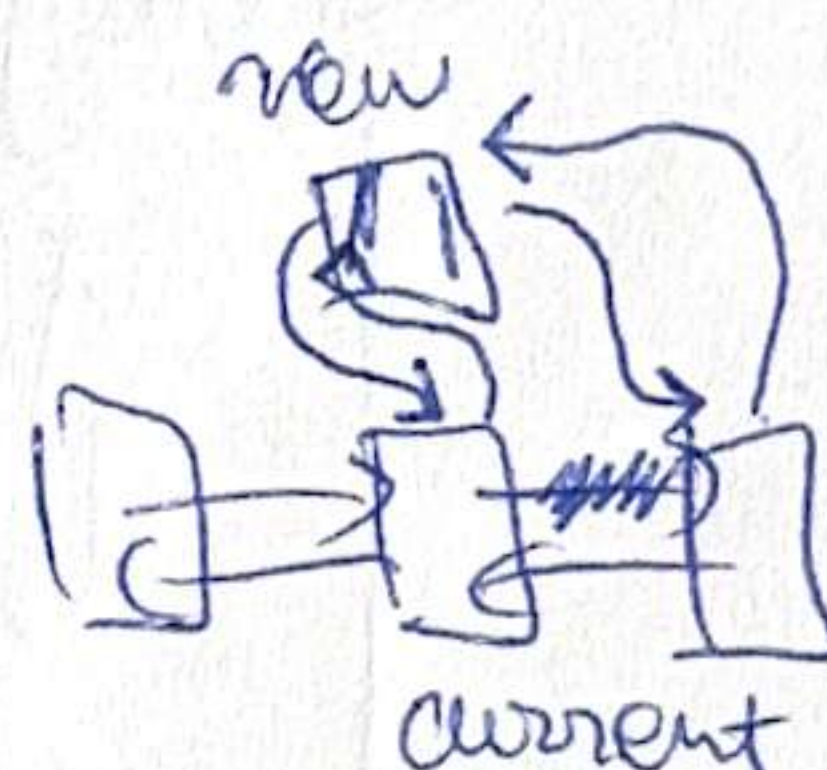
[newNode].info ← elem

[newNode].next ← [currentNode].next

[newNode].prev ← currentNode

[[currentNode].next].prev ← newNode

[currentNode].next ← newNode



Function `deleteElem(dll, elem)` is:

① iterate until elem found

`currentNode ← dll.head`

while `current ≠ Nil` and `[current].info ≠ elem` ex

`current ← [current].next`

`deleted ← current`

if `current ≠ Nil` then

 if `current = dll.head` then
 `deleteElem ← deleteFirst(dll)`

 else if `current = dll.tail` then
 `deleteElem ← deleteLast(dll)`

 else

`[current].next.prev ← [current].prev`

`[current].prev.next ← [current].next`

`[deleted].next ← Nil`

`[deleted].prev ← Nil`

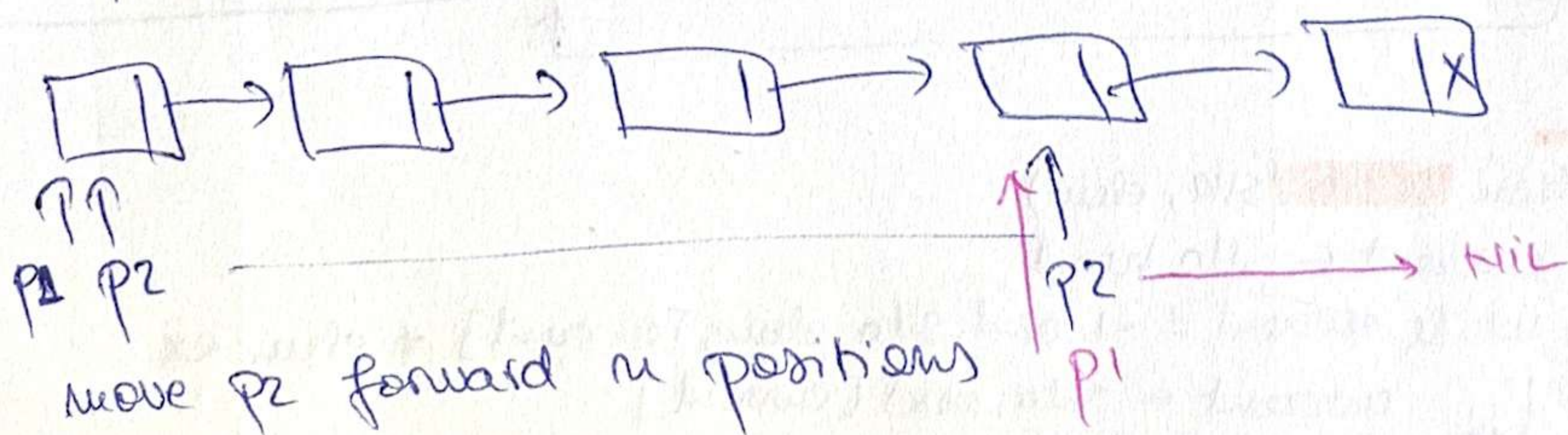
$O(n)$

`deleteElem ← deleted`

④ use search $\rightarrow O(n)$ $\rightarrow$ delete: $O(1)$ $\rightarrow$ search: $O(n)$ $\rightarrow O(n)$

2) n'th node from the end of a SL.

! 2 pointers with one traversal.



while `P2 ≠ Nil`

`P1 ← P1.next`

`P2 ← P2.next`

when `P2` reaches `Nil`, `P1` points to the n'th node from the end.

3) SSL

subalg init(ssl, rel) is:

```
ssl.head ← Nil  
ssl.rel ← rel
```

$\Theta(1)$

subalg insert(ssl, elem) is

newNode ← allocate()

[newNode].info ← Nil

if ssl.head = Nil then

| ssl.head ← newNode

else if ssl.rel(elem, [ssl.head].info) then

| [newNode].next ← ssl.head

| ssl.head ← newNode

else

em ← ssl.head

while [em].next ≠ Nil

$O(n)$

4) circular list.

→ no node has Nil as next

lectures

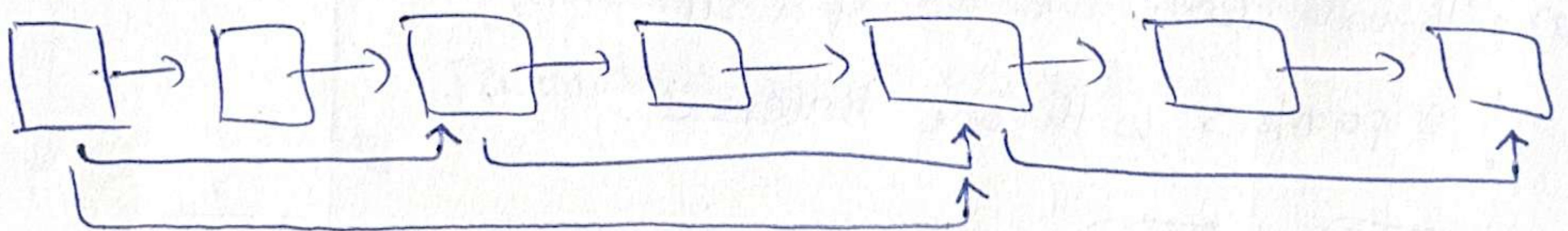
1) XOR LL

1 → 5 ...

2) Skiplist

1 → 1 → 1 → ...

1 → 3 ...



3) SLA

function search(sla, elem)

current ← sla.head

while current ≠ -1 and sla.elem[current] ≠ elem ex

| current ← sla.next[current]

if current ≠ -1 then

| search ← True

else search ← False

$O(n)$

Subalg **init(sll)** is

$sll.cap \leftarrow INIT_CAPACITY$

$sll.elem \leftarrow @array$

$sll.next \leftarrow @array$

$sll.head \leftarrow -1$

for $i \leftarrow 1, sll.cap - 1$ ex

$sll.next[i] \leftarrow i + 1$

$\Theta(m)$

$\hookrightarrow$ initial capacity.

$sll.next[sll.cap] \leftarrow -1$

$sll.firstEmpty \leftarrow 1$

Subalg **insertFirst(sll, elem)** is

if $sll.firstEmpty = -1$ then

$\Theta(1)$ am.

Lecture 7

1) Delete elem in SLLA.

Subalg deleteElem(SLLA, elem) is:

```
modC ← SLLA.head
prevNode ← -1
while modC ≠ -1 and SLLA.elements[modC] ≠ elem ex
    prevNode ← modC
    modC ← SLLA.next[modC]
if modC ≠ -1 then
    if modC = SLLA.head then
        SLLA.head ← SLLA.next[SLLA.head]
    else
        SLLA.next[prevNode] ← SLLA.next[modC]
    SLLA.next[modC] ← SLLA.firstEmpty
    SLLA.firstEmpty ← modC
else
    @ 7
```

$O(n)$

2) DLLA

(one array, node struct)

- allocate()
- free()
- subalg insertPositionC() $\rightarrow O(n)$

subalg init(it, dlla) is

```
it.list ← dlla
it.currentElement ← dlla.head
```

$O(1)$

subalg getCurrent(it) is:

```
if it.currentElement = -1 then
    @ throw
    @ 7
    getCurrent ← it.list.nodes[it.currentElement].info
```

$O(1)$

subalg next(it) is:

```
if it.currentElement = -1 then
    @throw
else
    it.currentElement ← it.list.nodes[it.currentElement].next
```

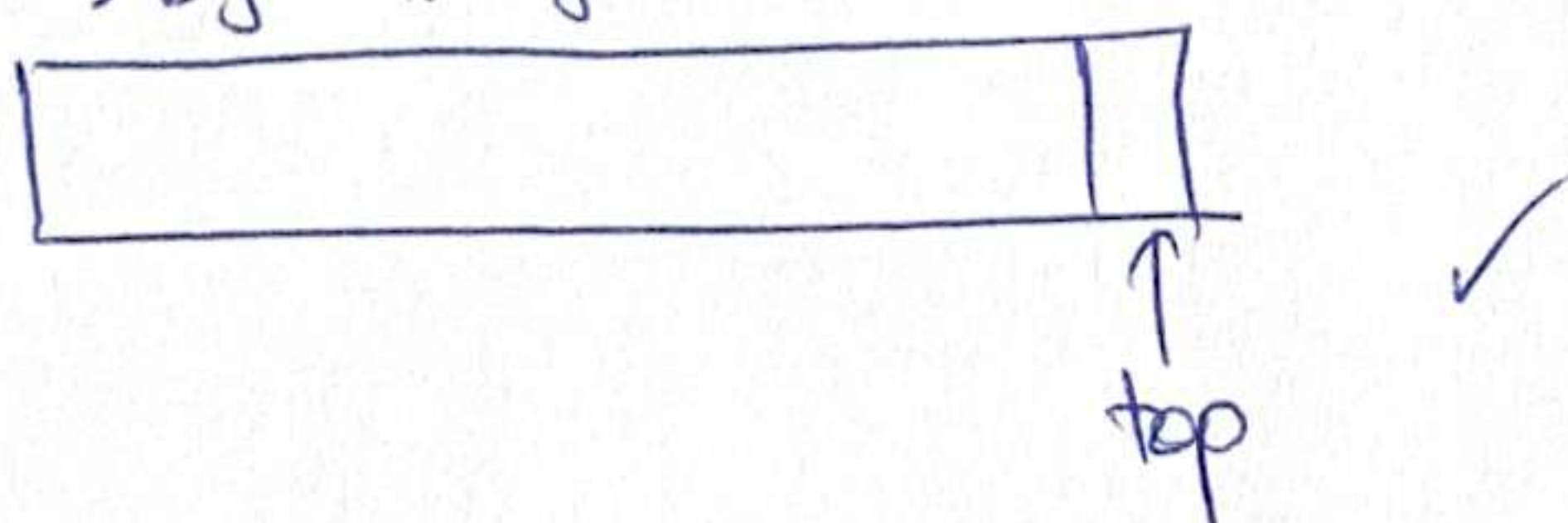
$\Theta(1)$

function valid(it) is:

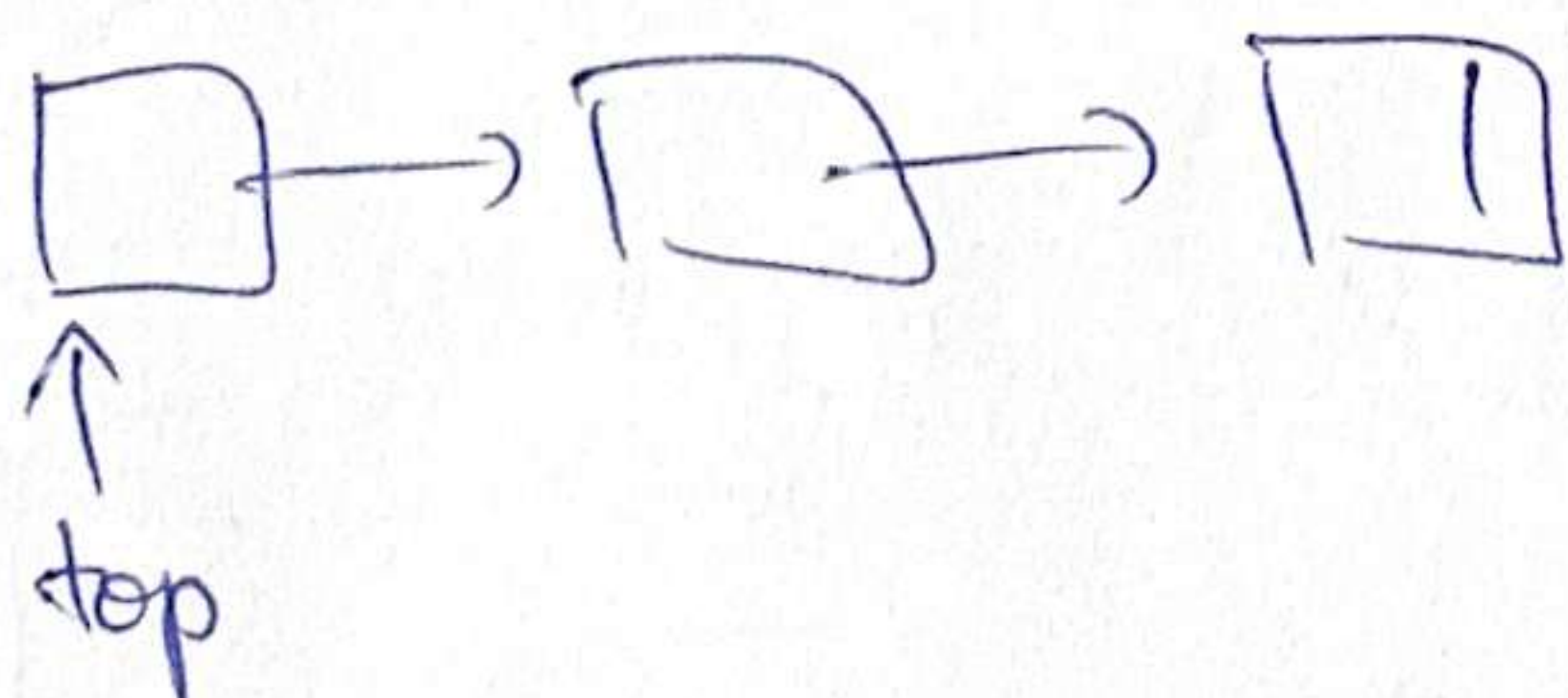
```
if it.currentElement = -1 then
    valid ← False
else
    valid ← True
```

$\Theta(1)$

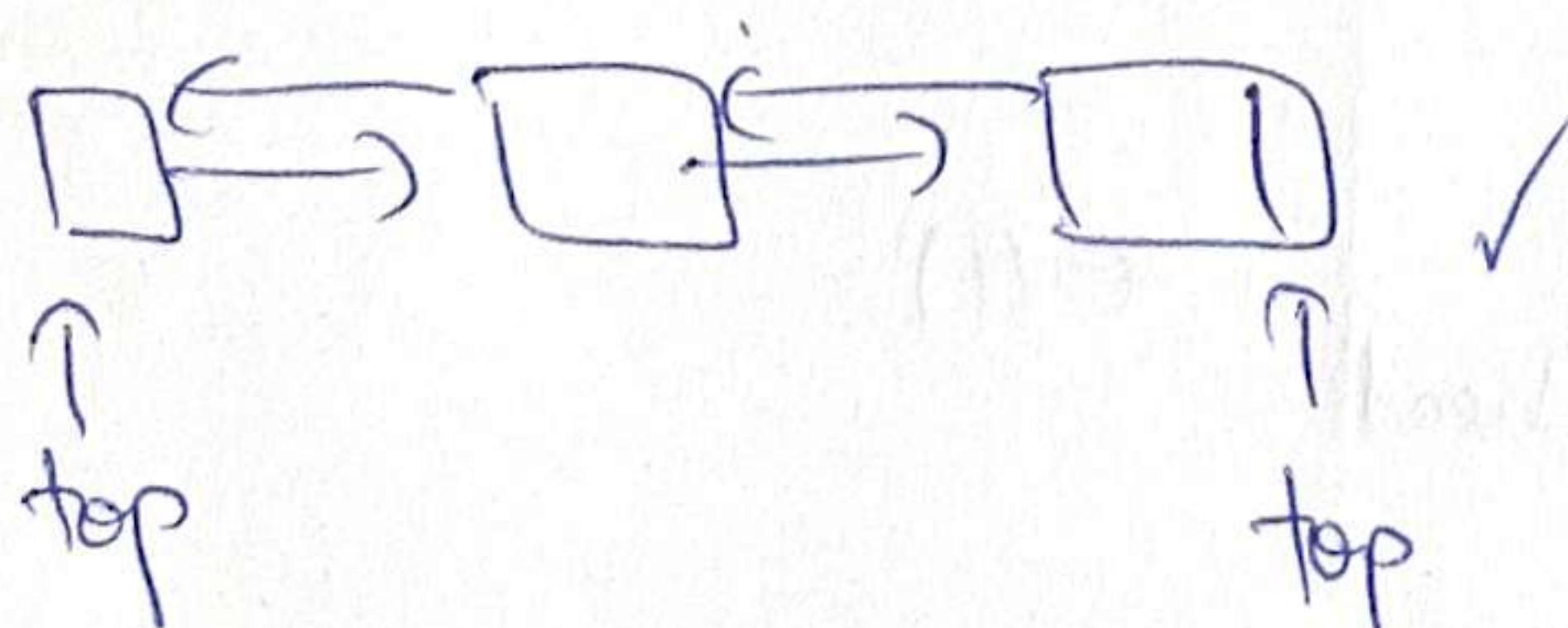
3) Stack
→ Dyn-Array



→ SL

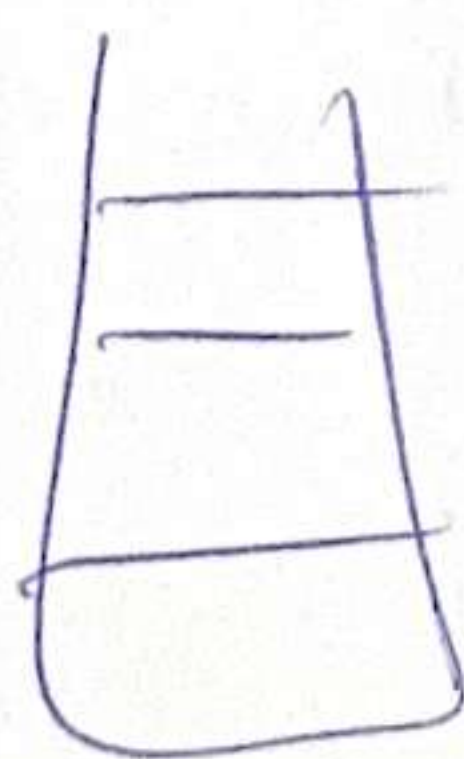


→ DL

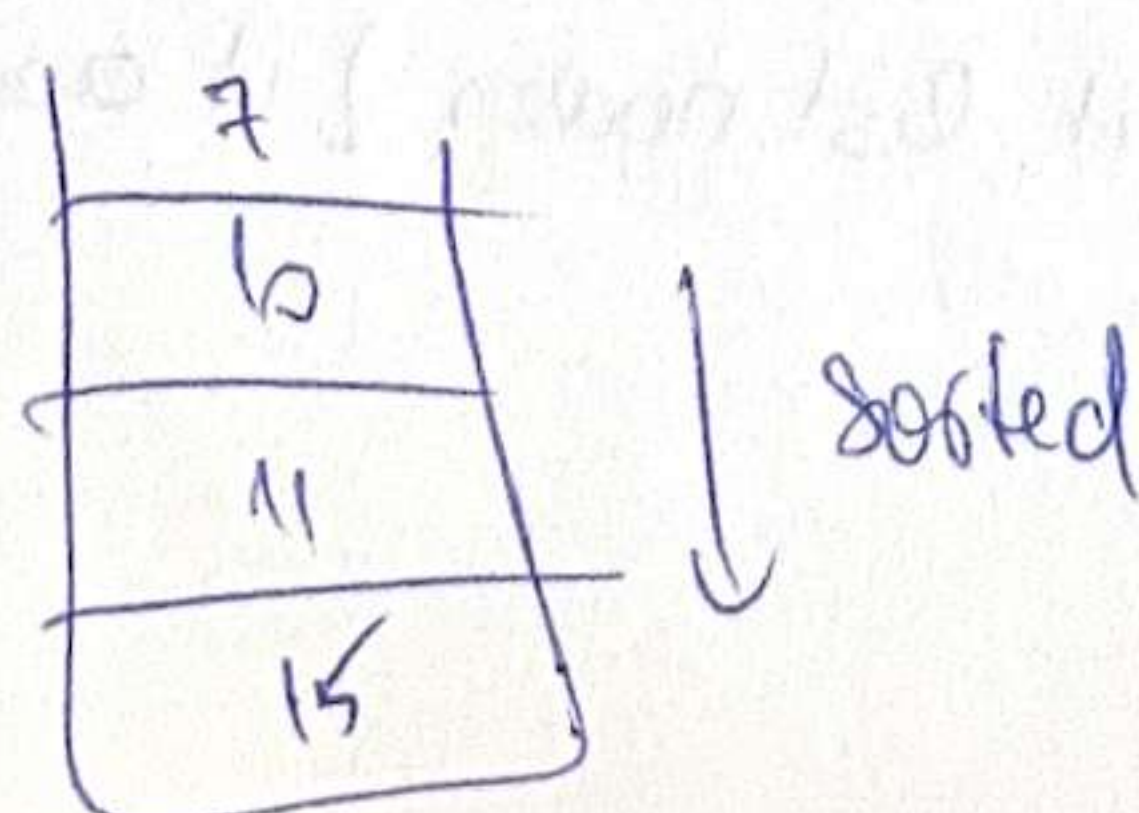


4) Special Stack

Orig.

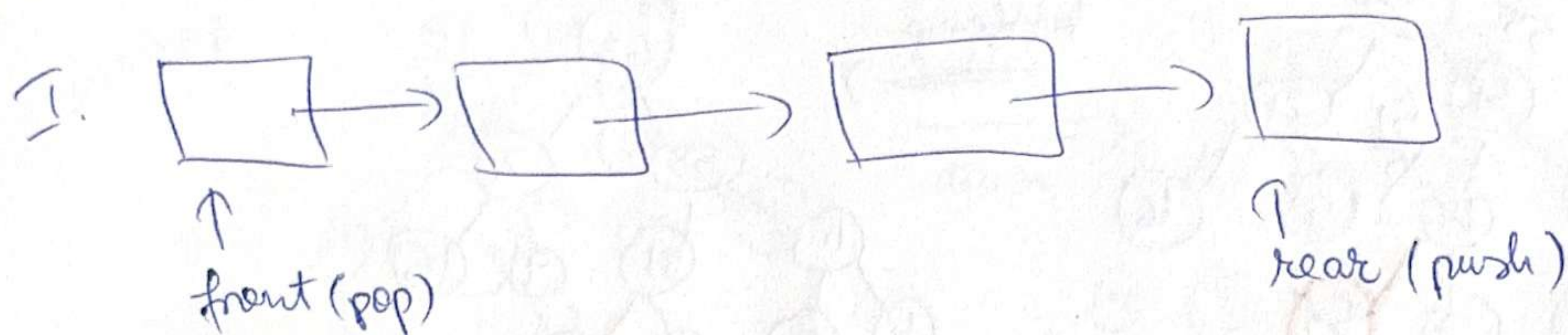


Min Stack

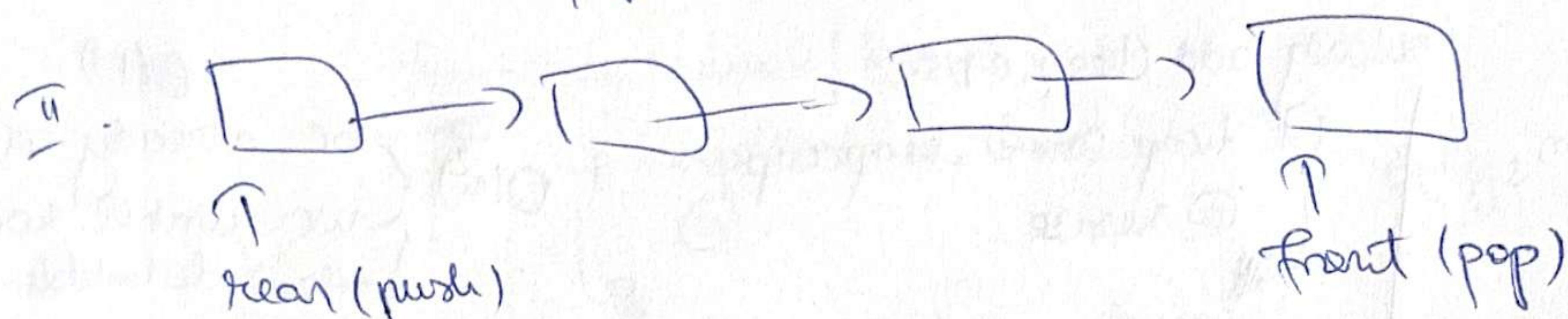


Lecture 8

1) Queue on SL



⇒ Better if we pop more.



⇒ Better if we push more.

I. is better because

2) Queue on DL → not recommended.

3) Queue on DA of fixed-size arrays?

4) PriorityQueue

→ I. Sorted by priority

⇒ push: $O(n)$

⇒ pop: $O(1)$

⇒ top: $O(1)$

→ II. By order of insertion:

push: $O(1)$

pop: $O(n)$

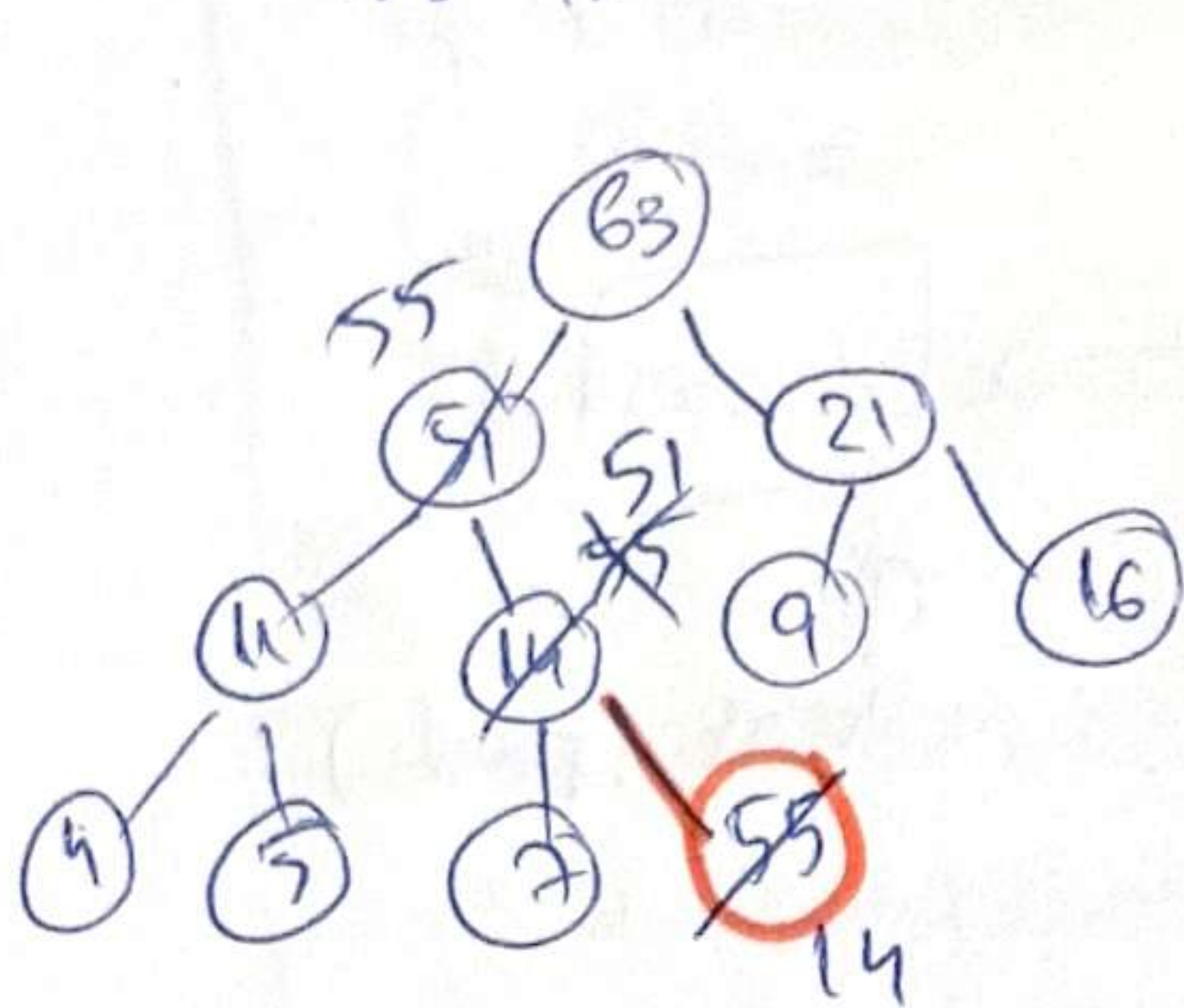
top: $O(n)$

5) Bin. heap → efficient repres. for PrQueue

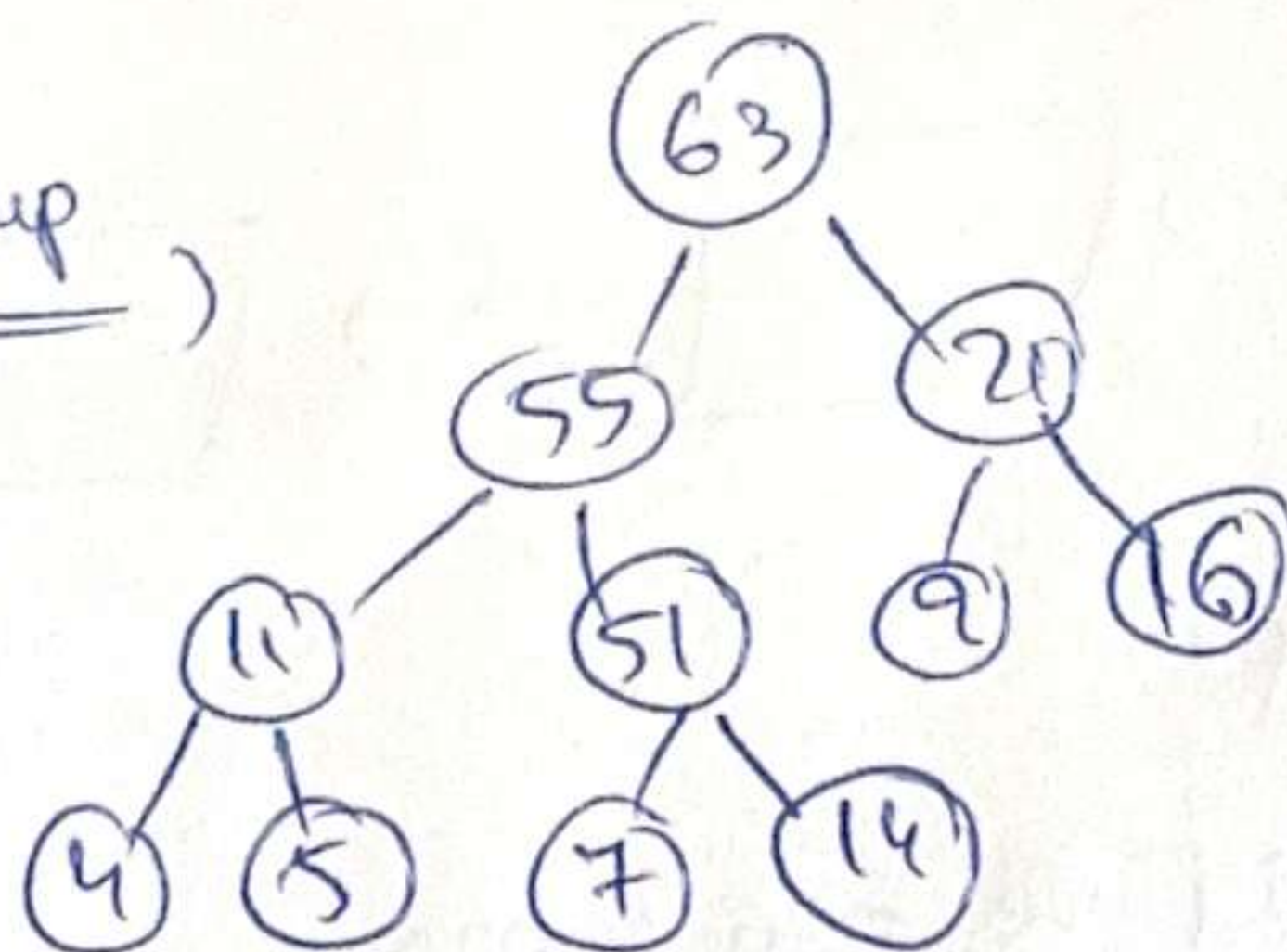
↳ structure (no gaps)

↳ property $\begin{cases} \text{min} \\ \text{max} \end{cases}$

ADD new node in heap (55)



bubble-up



subalg add(heap, e) is :

if heap.len = heap.cap
@ resize

heap.elements[heap.len+1] ← e

heap.len ← heap.len + 1

bubble-up(heap, heap.len)

$O(\lg n)$ BC: already sorted
WC: until root
do bubble-up
AC -

subalg bubble-up(heap, p) is:

pos ← p

elem ← heap.elements[p]

parent ← p/2

while pos > 1 and elem > heap.elements[parent] ex

heap.elements[pos] ← heap.elements[parent]

pos ← parent

parent ← pos/2

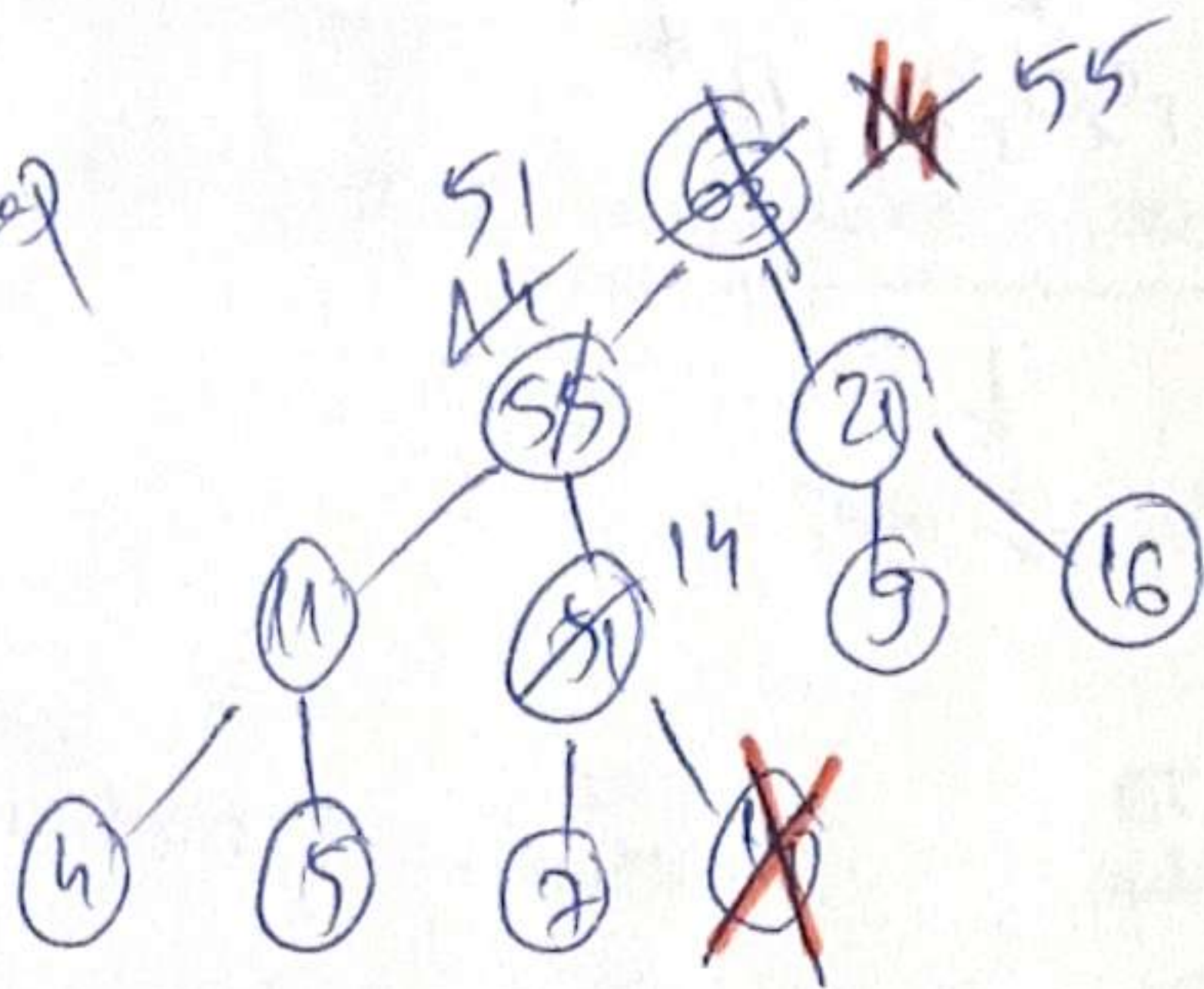
$O(\lg_2 n)$

heap.elements[pos] ← elem

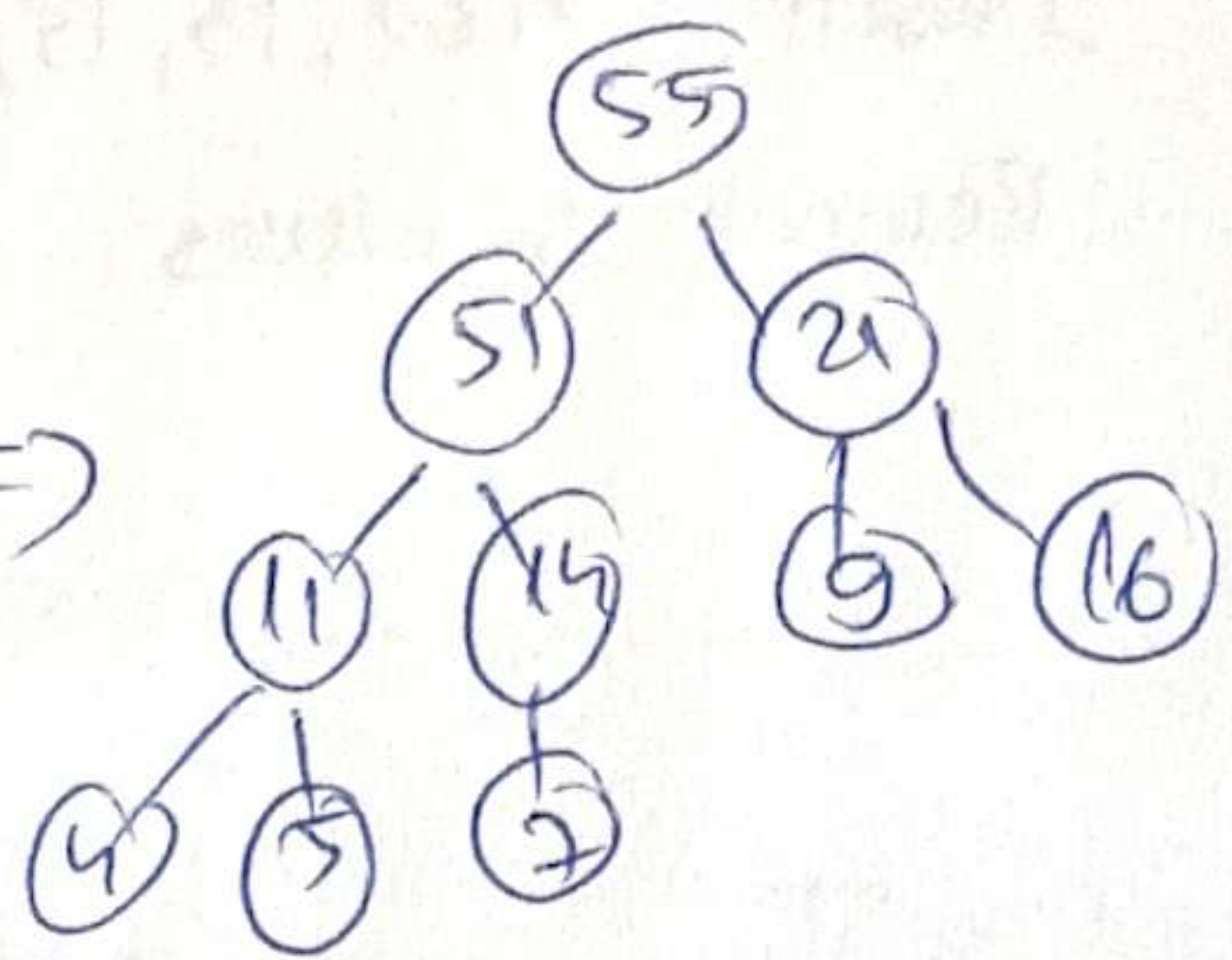
"max heap" property

REMOVE (root.)

max heap



bubble
down



function remove(heap) is:

if heap.len = 0 then
@---

deletedElem ← heap.elem[1]
heap.elem[1] ← heap.elem[heap.len]
heap.len ← heap.len - 1
bubble-down(heap, 1)
remove ← deletedElem

$O(\log_2 n)$ BC $\Theta(1)$
WC $O(\log_2 n)$

Subalg bubble-down(heap, p) is:

pos ← p
elem ← heap.elem[pos]
while pos < heap.len ex
 maxChild ← -1
 if pos * 2 ≤ heap.len then // left ∃
 maxChild ← pos * 2
 if pos * 2 + 1 ≤ heap.len and

max elem in max heap: $\Theta(1)$, elem[0]

min elem in max heap: $\Theta(n)$, leaf;

↳ Improve: minElem variable → update it

Ex ① Max-heap.

Insert: 8, 27, 13, 15^{*}, 32, 20, 12, 50^{*}, 29, 11^{*}.

Remove 3 elems.

~~Ex ②~~:

Ex ②: Min-heap: 15, 17, 9, 11, 5, 19, 7 (insert).

remove all elems

HeapSort

→ min heap → add elems to it

→ remove elems from it

$O(n \log_2 n)$ time

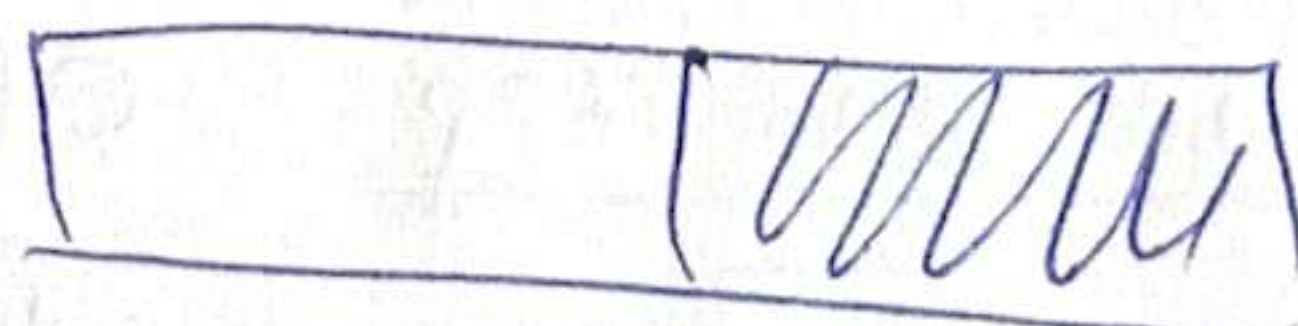
$\Theta(n)$ space

→ max heap: → space optimisation.

→ split array in two:

⇒ $O(n)$

⇒ remove: $O(n \log_2 n)$



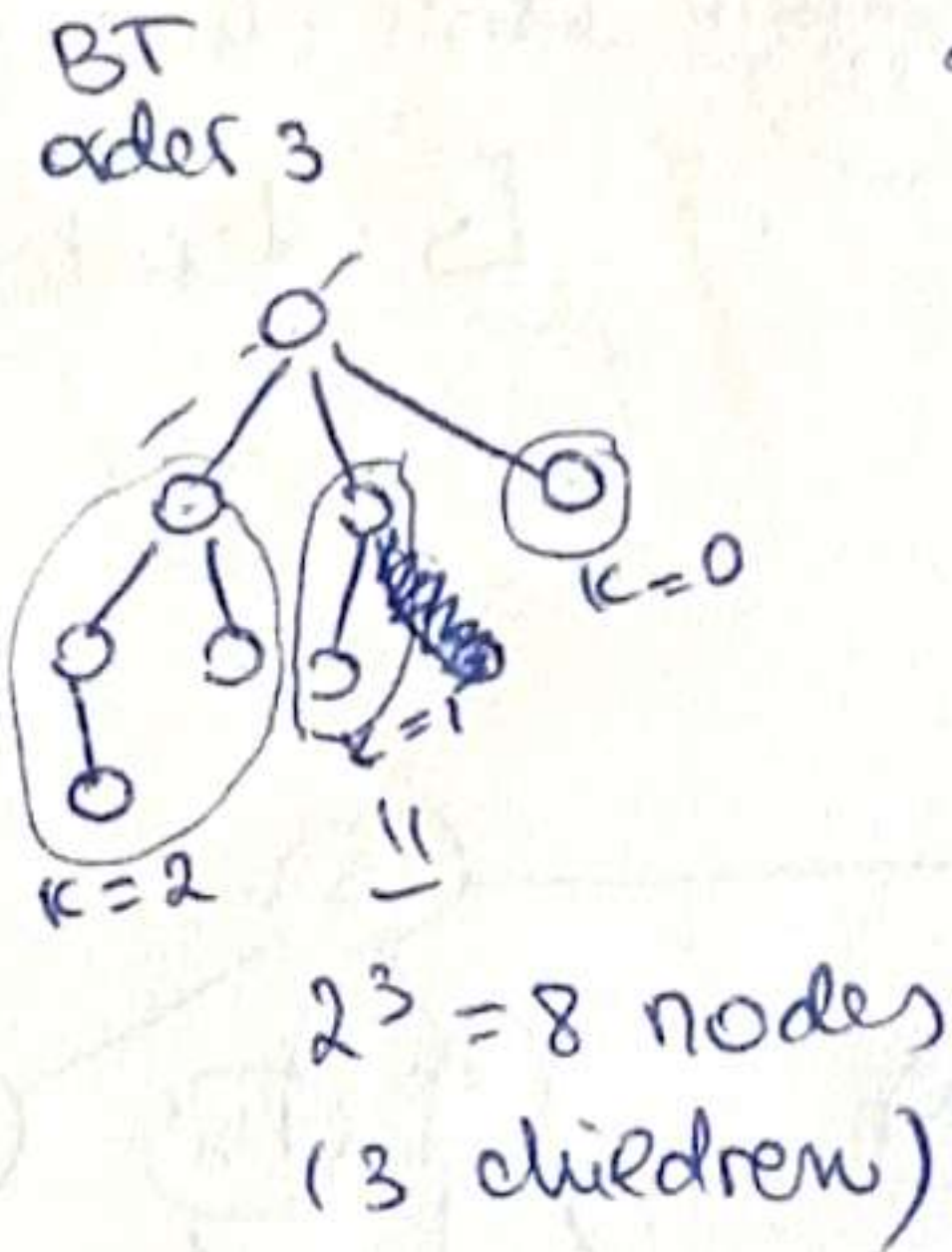
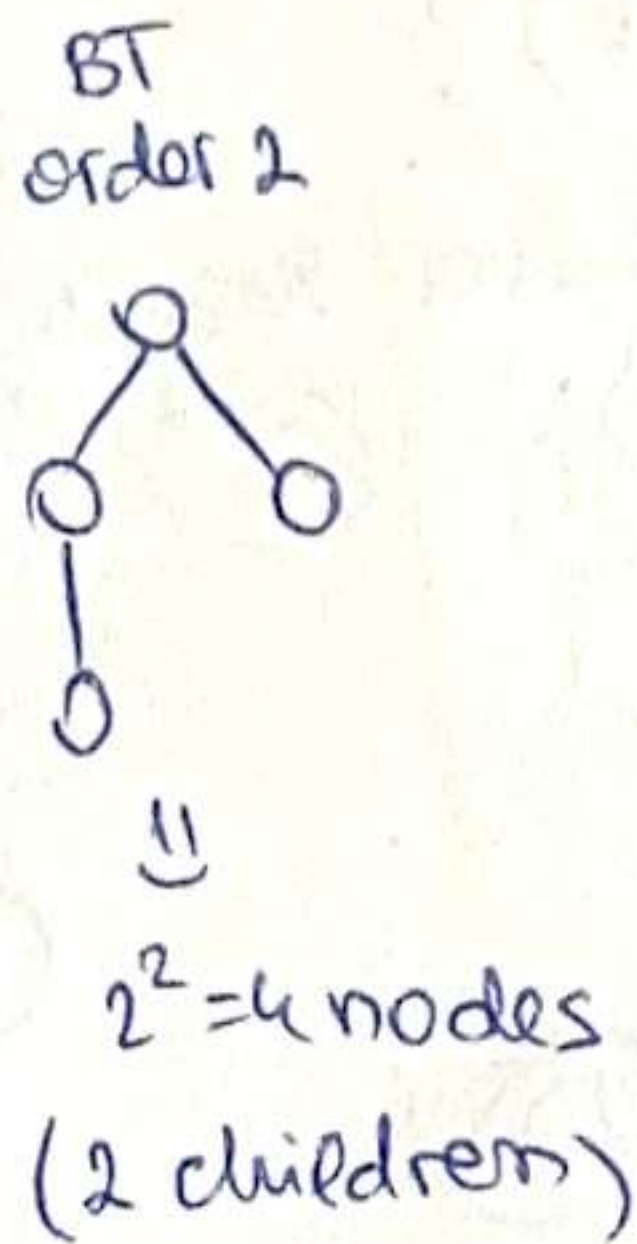
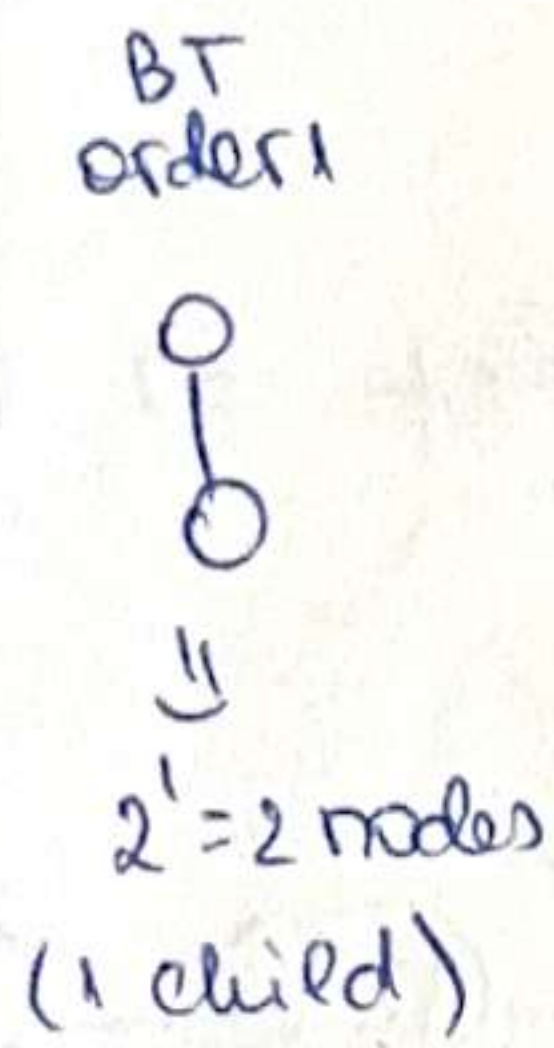
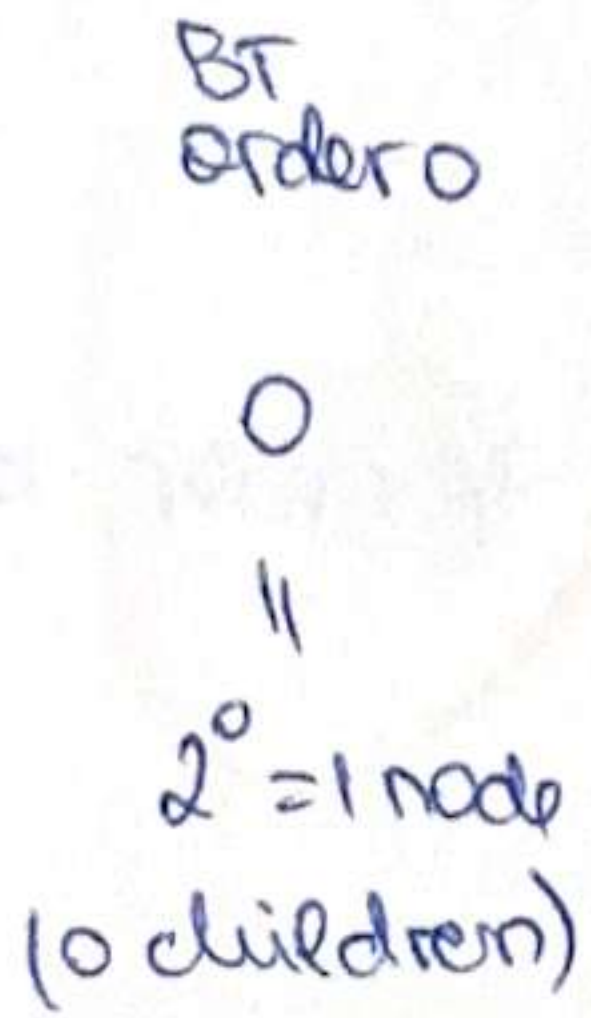
← leaves

bubble-down for (*) element

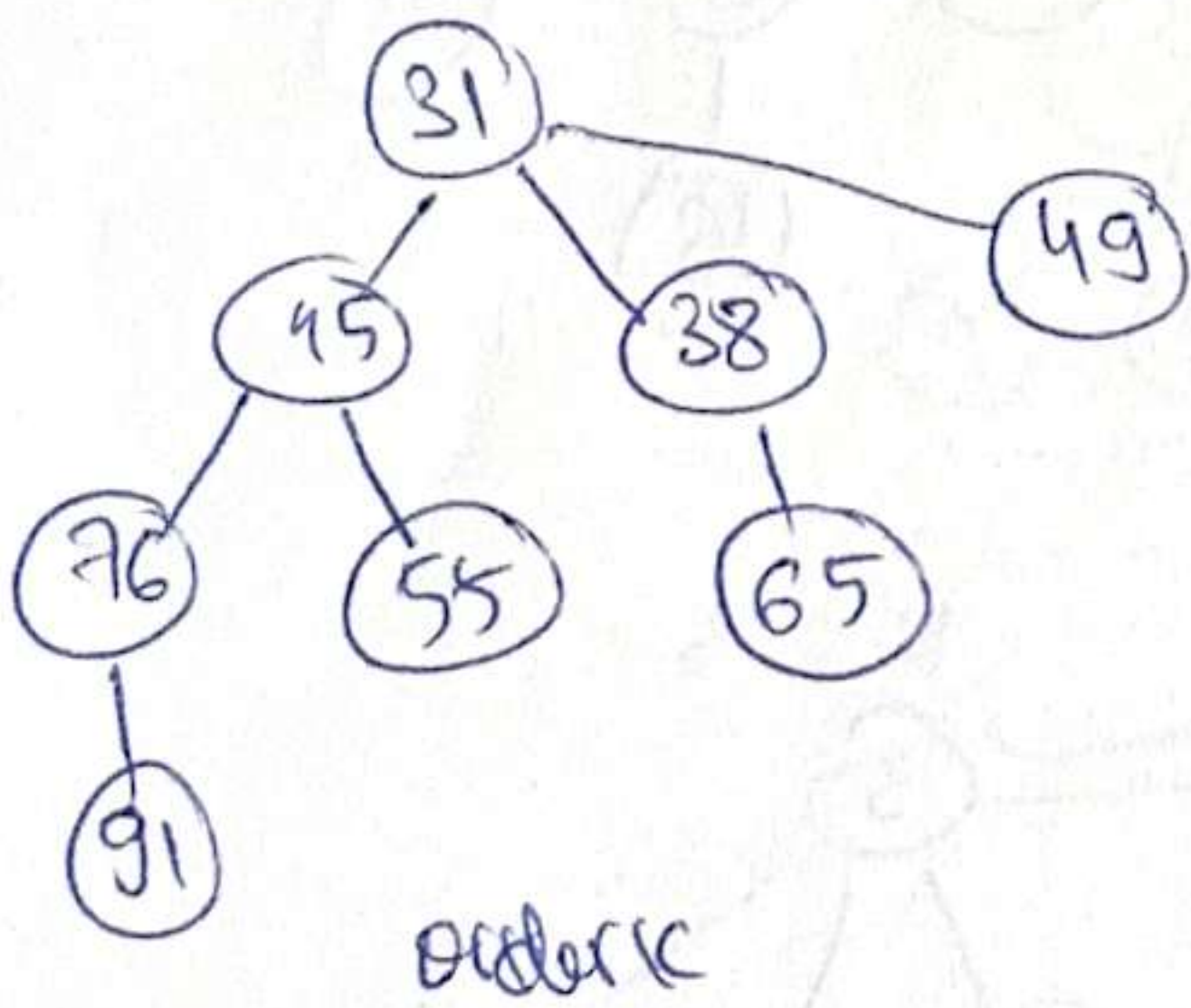
Lecture 9

1) Bimomial heap
= collection of bimomial trees

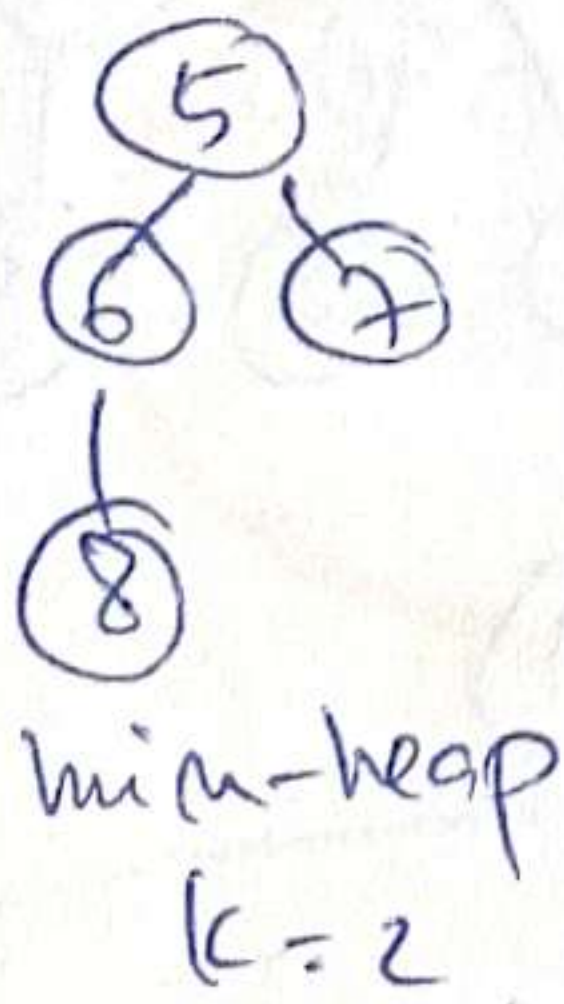
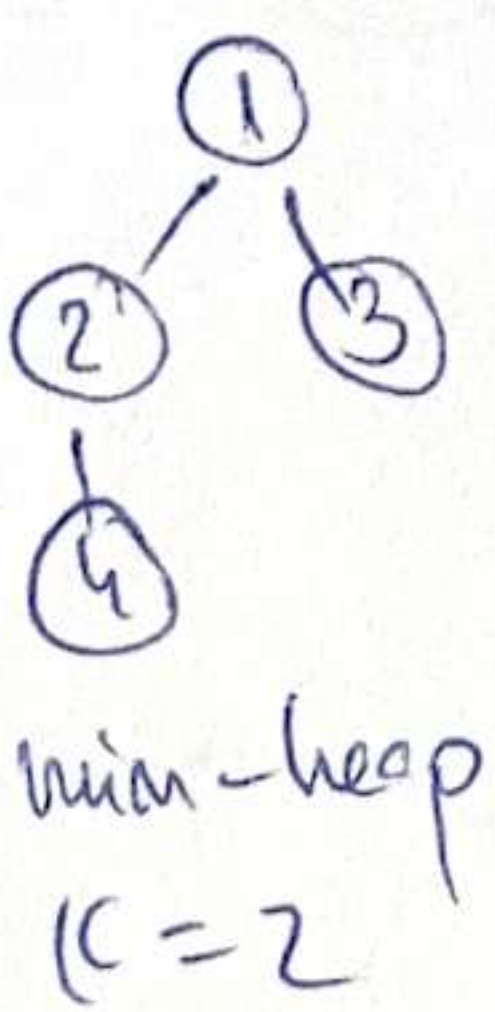
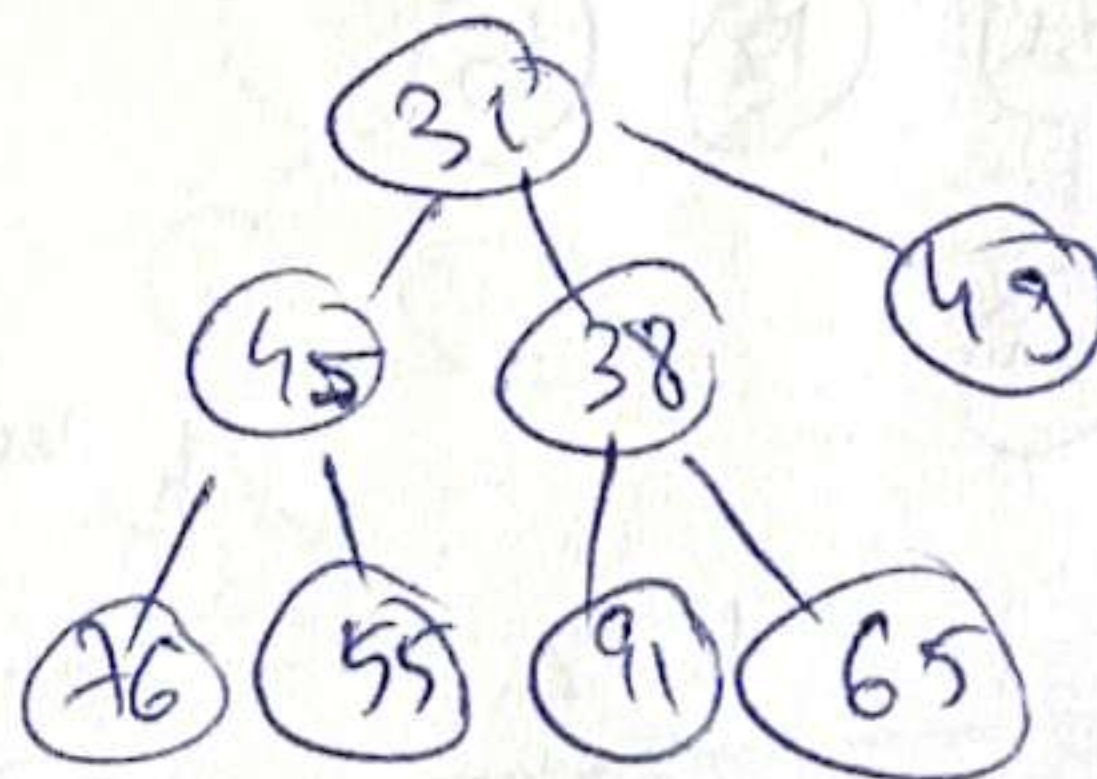
↓
can be defined recursively



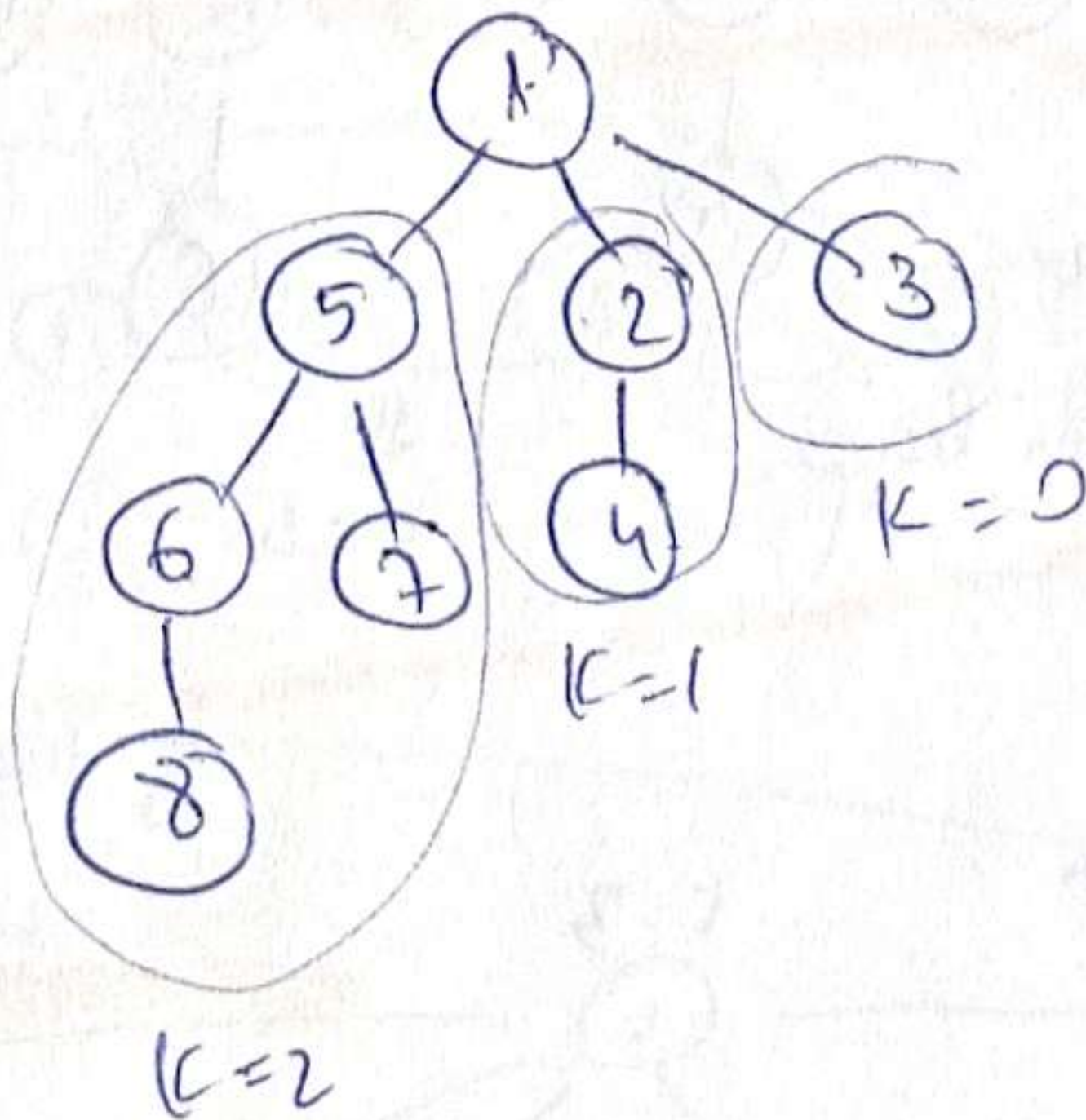
If we remove the root
↓
2 valid BTs.



NOT

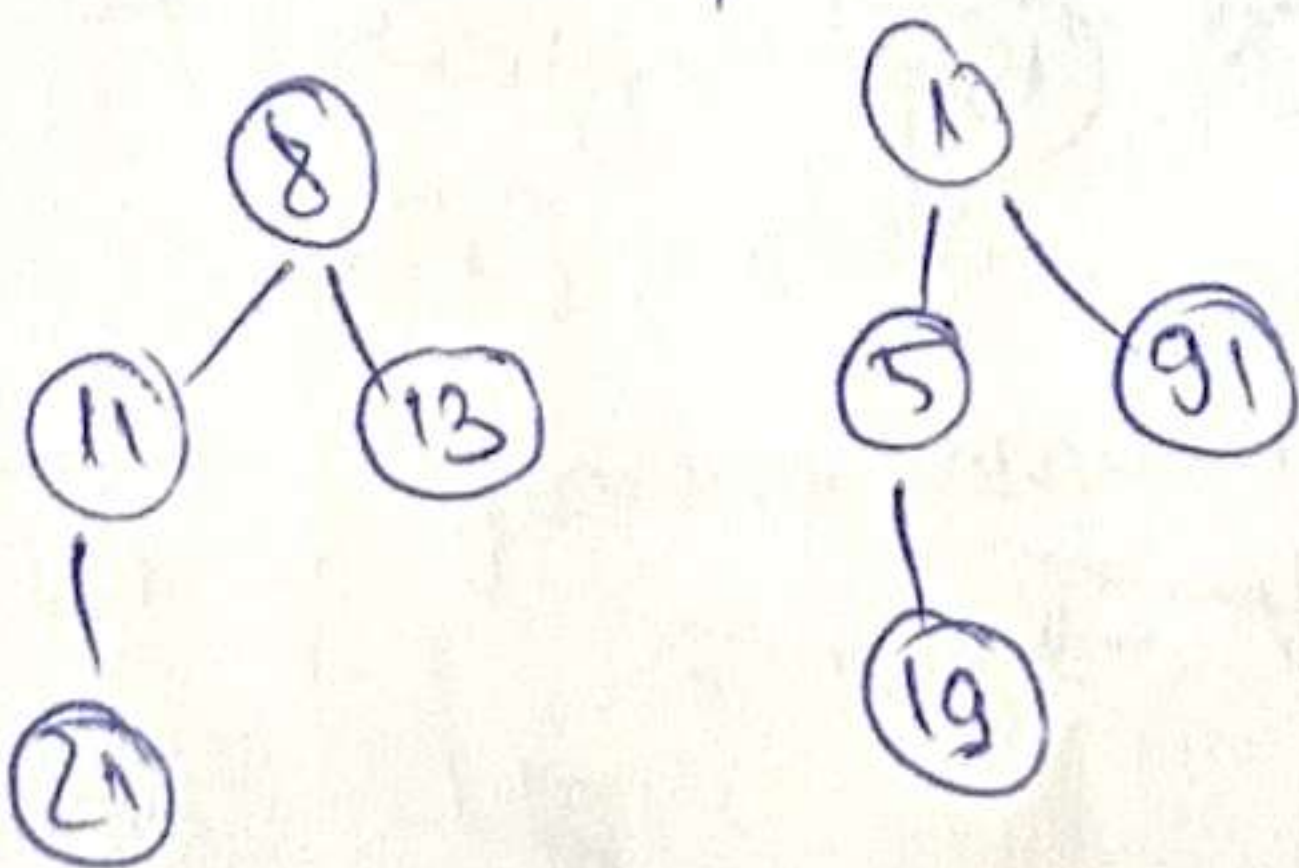


⇒

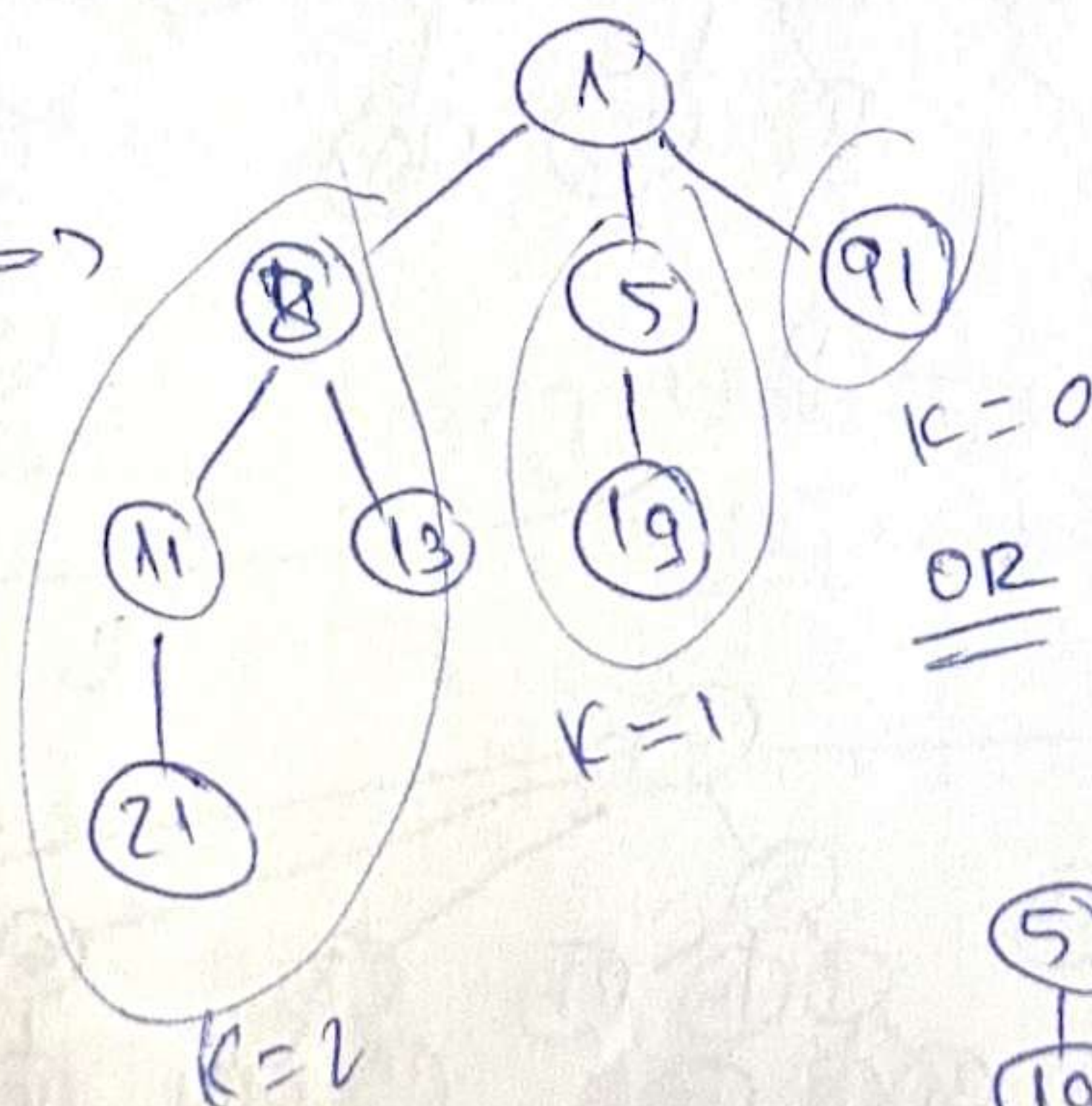


min
heap prop.
respected ✓

merge (trees)

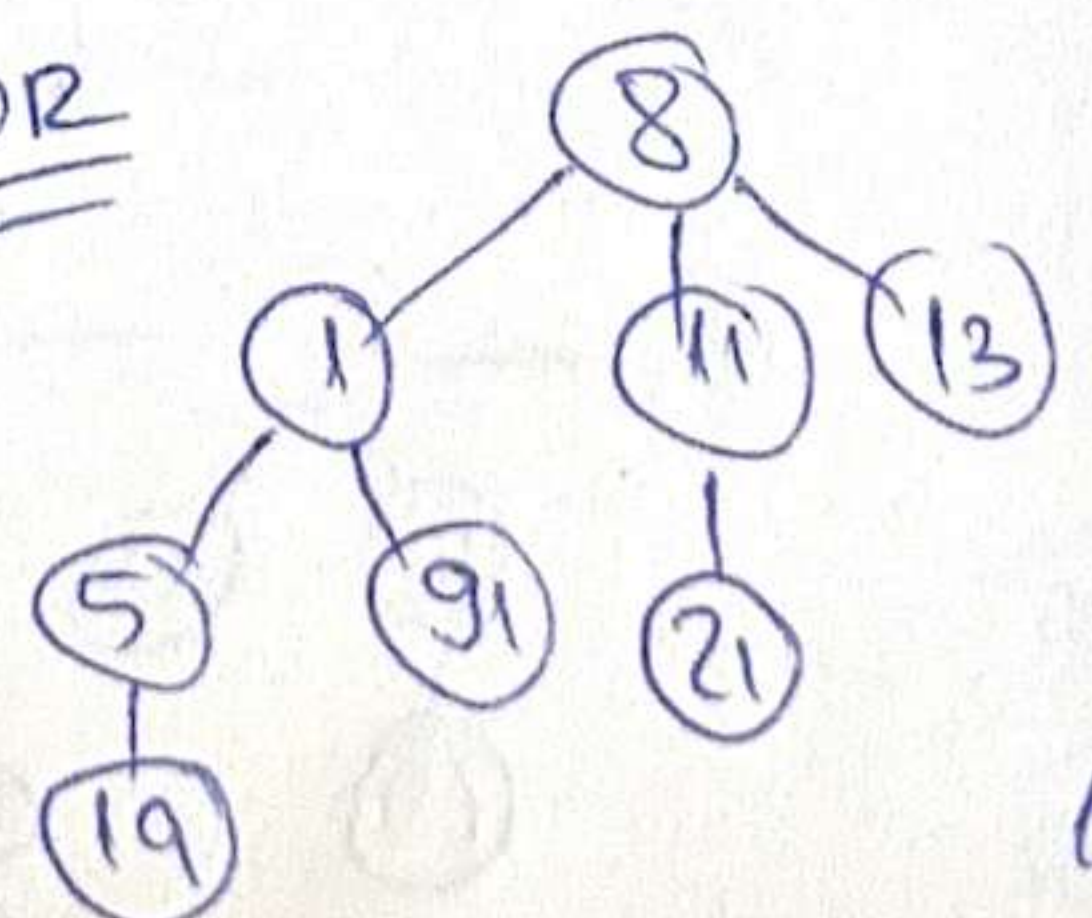


⇒



$k=0$

OR



M nr elems.

BT orders

$$M=14 = 1110 = 2^3 + 2^2 + 2^1 \Rightarrow$$

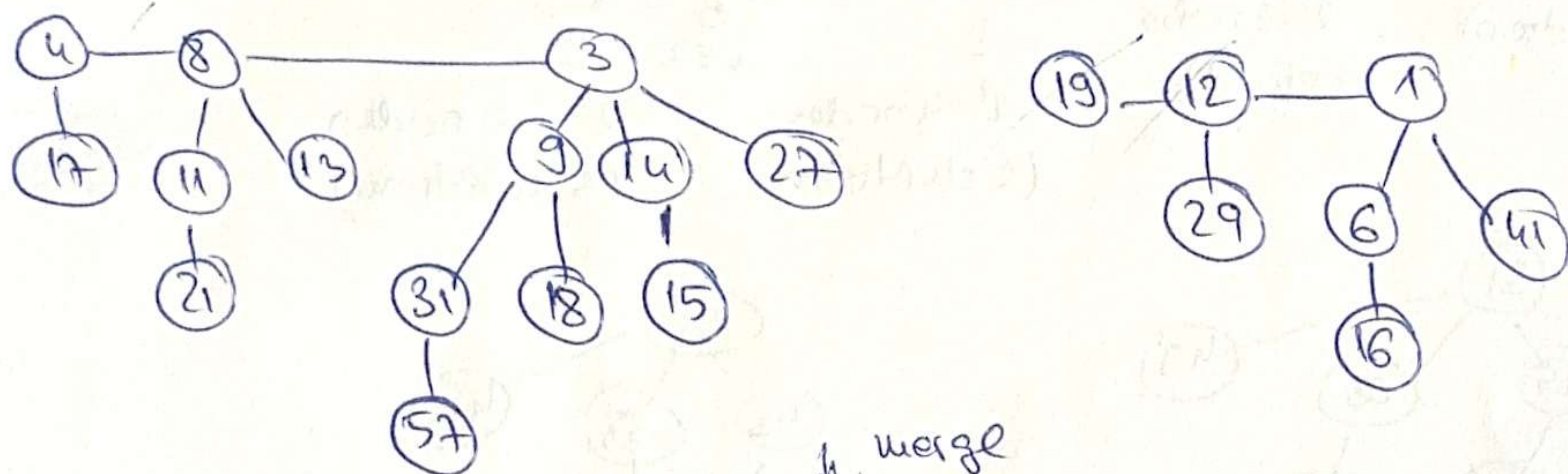
a binomial ~~tree~~ ^{heap} with 14 nodes contains binomial trees of orders 3, 2, 1, but sorted in reverse: 1, 2, 3.

Merge binomial heaps = merge sorted LL

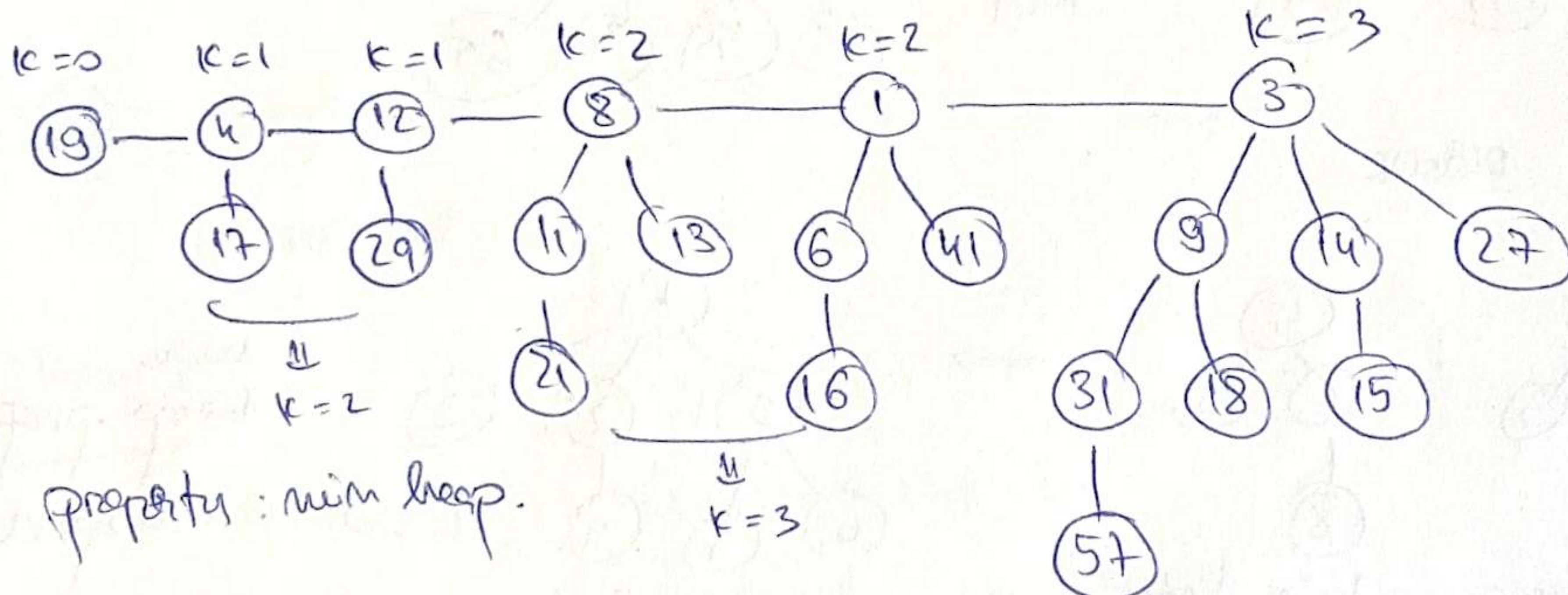
$\Rightarrow$ same order check (k)?

$\hookrightarrow$ iterate over resulting list $\Rightarrow$ BT of order $k+1$

Merge heaps

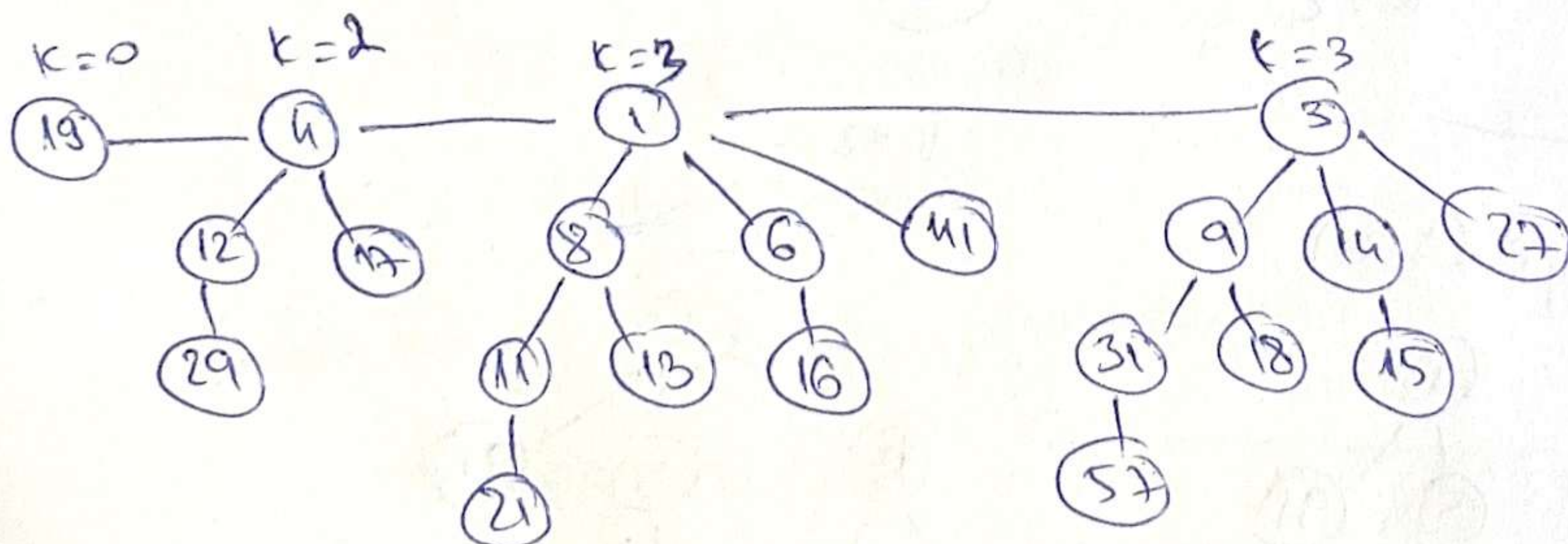


$\hookrightarrow$ merge

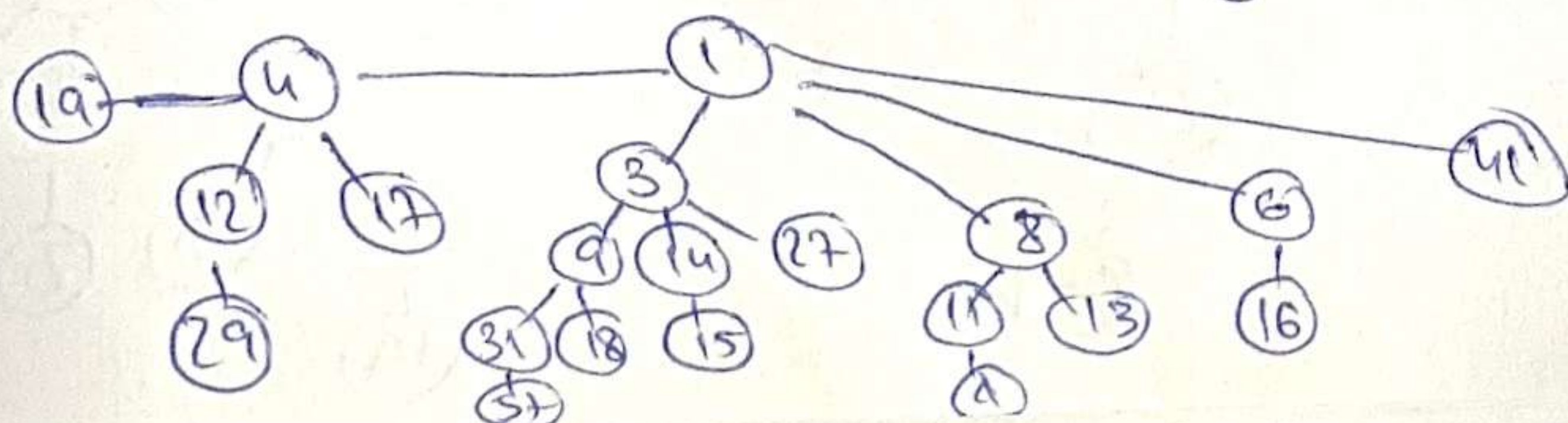


$\hookrightarrow$ merge

$\hookrightarrow$ merge

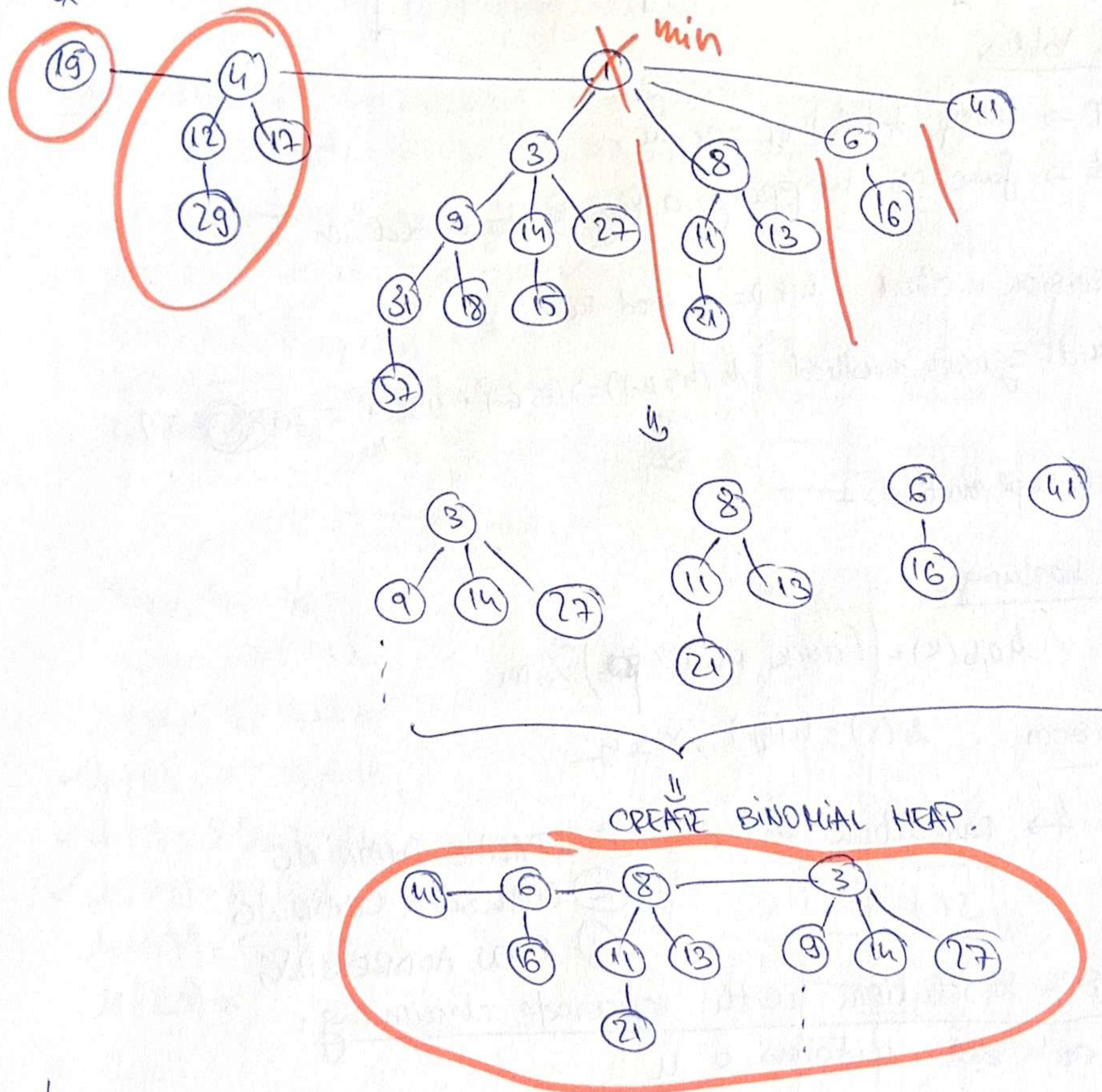


$\hookrightarrow$



Pop = remove min. elem. = remove root of one of the bin. trees $\Rightarrow$
 $\Rightarrow$ seq. of binomial trees $\Rightarrow$ transform in bin. heap. $\Rightarrow$ merge
 between this new heap and ... ?

ex:



$\Downarrow$ merge with the rest.

2) Direct address tables. (fast $\Rightarrow$ positions = array) $\rightarrow \Theta(1)$
 $\rightarrow$ 4B (-)
 $\rightarrow$ 1.

T = array with m positions

$T[k] = \text{data}$

function search(T, k) is

search $\leftarrow T[k]$



subalg insert(T, x) is

$T[\text{key}(x)] \leftarrow x$



subalg delete(T, x) is:

$T[\text{key}(x)] \leftarrow \text{Nil}$



(21)
 (21)

Insert

↳ slot $\rightarrow$ empty $\Rightarrow$ create node.

$\Theta(1)$ WC.

↳ occupied $\Rightarrow$ create node $\Rightarrow$ add to beginning of LL.

⊕ search if $\exists$ already $\Rightarrow$ 0

Ex: HT, $m=6$, sep chaining. $\alpha = \frac{n}{m} \geq 0.7 \Rightarrow$ double size ($m \times 2$).
div. method, add: 38, 11, 8, 72, 57, 29, 2.

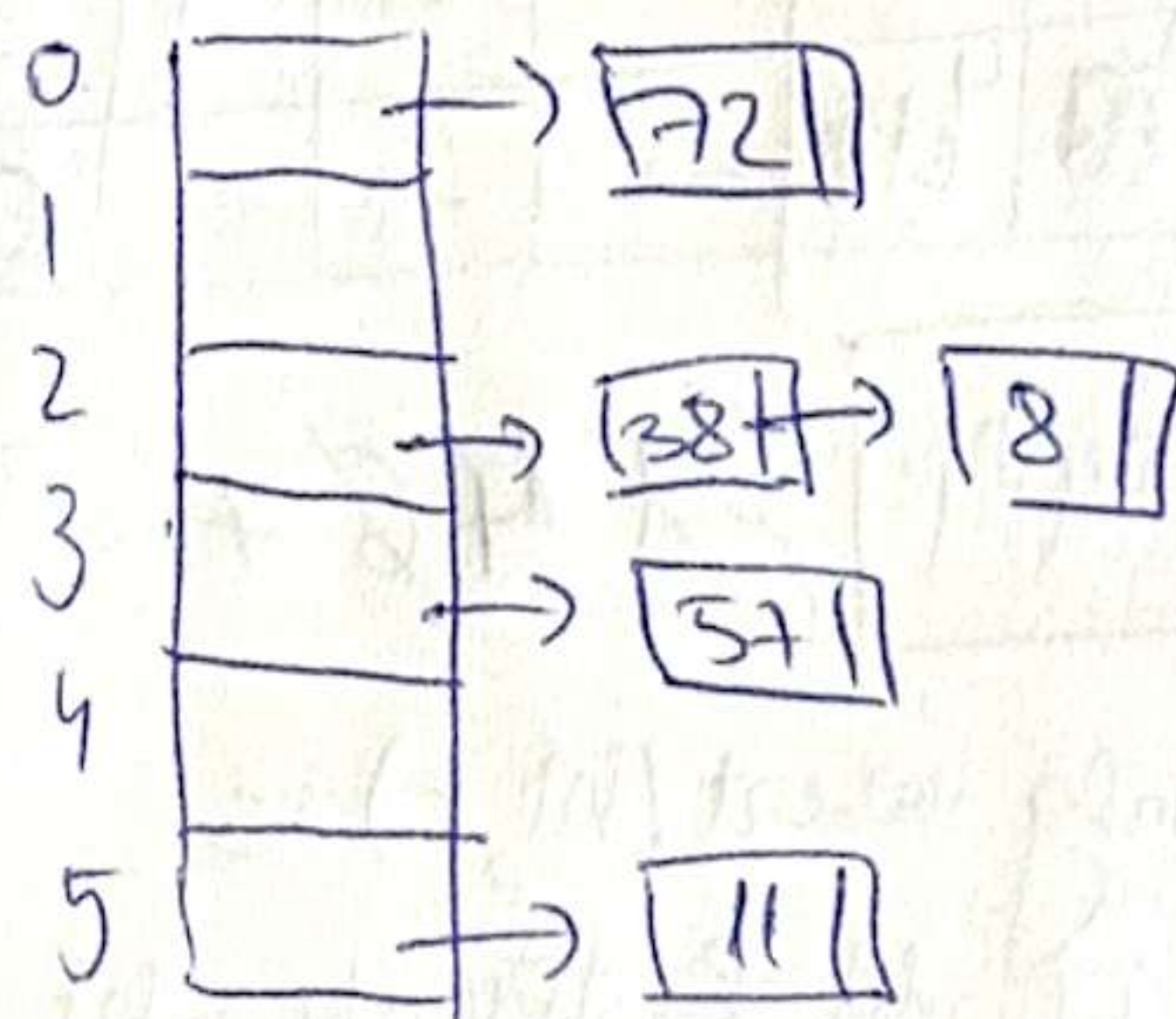
$$h(38) = 38 \% 6 = 2, \alpha = \frac{1}{6} < 0.7 \checkmark$$

$$h(11) = 5 \rightarrow \alpha = \frac{2}{6} < 0.7 \checkmark$$

$$h(8) = 2 \rightarrow \alpha = \frac{3}{6} < 0.7 \checkmark$$

$$h(72) = 0; \alpha = \frac{4}{6} < 0.7 \checkmark$$

$$h(57) = 3; \alpha = \frac{5}{6} > 0.7 \text{ STOP}$$



resize, rehash:

$$\Rightarrow m = 6 \times 2 = 12.$$

$$h(38) = 38 \% 12 = 2$$

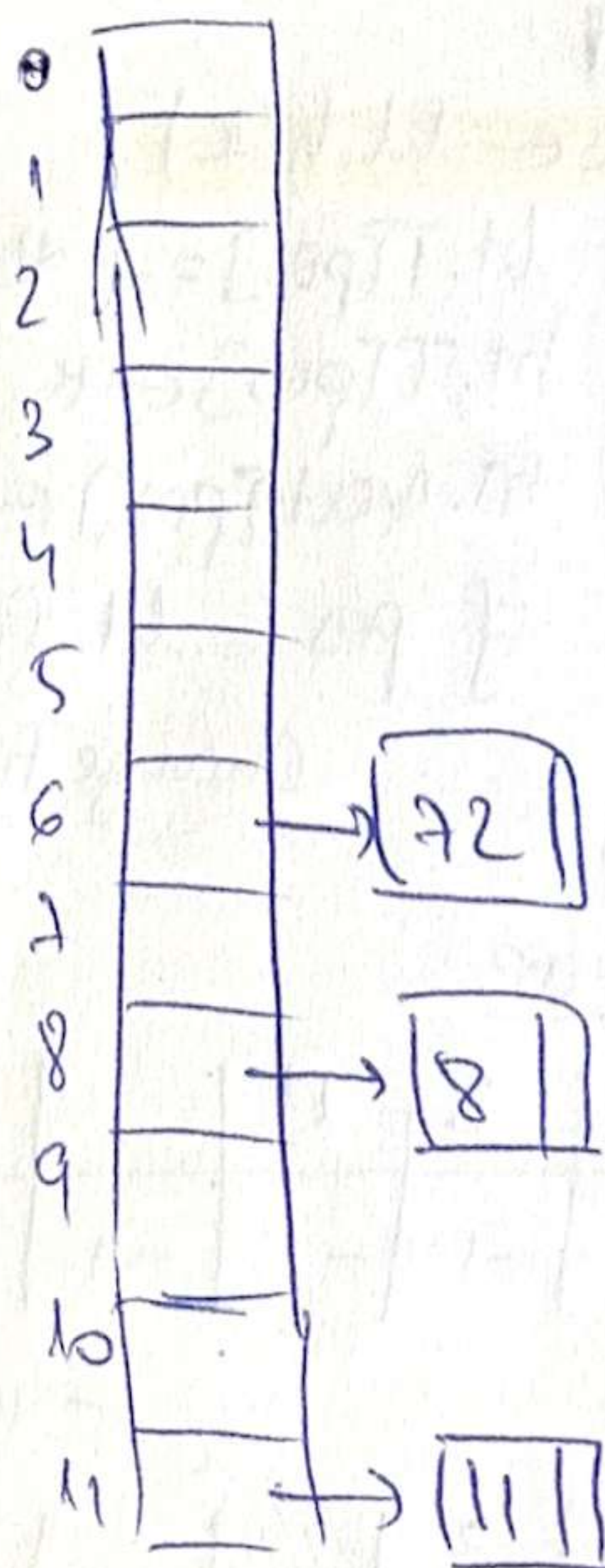
$$\checkmark h(11) = 11 \% 12 = 11$$

$$\checkmark h(8) = 8 \% 12 = 8$$

$$\checkmark h(72) = 72 \% 12 = 0$$

$$h(57) = 57 \% 12 = 9$$

$$h(29) = 5$$



Lecture 10

1) HT Iterator for separate chaining

HTIterator:

ht: HT

currentPos: Int

currentNode: TNode

- valid

- get current

- next

subalg

init (ith, ht) is

ith.ht $\leftarrow$ ht

ith.currentPos $\leftarrow$ 0

$\Theta(m)$

while ith.currentPos $<$ ht.m and

ht.T[ith.currentPos] $\neq$ Nil

ith.currentPos $\leftarrow$ ith.currentPos + 1

if ith.currentPos $<$ ht.m then

ith.currentNode $\leftarrow$ ht.T[ith.currentPos]

else
ith.currentNode $\leftarrow$ Nil

2) Coalesced chaining ($m=13$)

Insert: 5, 18, 16, 15, 13, 31, 26

$$h(5)=5 \checkmark \quad h(16)=3 \checkmark \quad h(13)=5 \checkmark \quad h(26)=0 \checkmark$$

$$h(18)=5 \checkmark \quad h(15)=2 \checkmark \quad h(31)=5 \checkmark$$

$\swarrow 13 \leftarrow 26$
 $\swarrow 18 \leftarrow 31$

pos	0	1	2	3	4	5	6	7	8	9	10	11	12
T	18	13	15	16	31	5	26						
next	18	13	-1	-1	-1	⊖	-1	-1	-1	-1	-1	-1	-1

firstEmpty: ~~0~~ ~~1~~ ~~4~~ ~~6~~ 7

Subalg insert(ht, k) is

if $ht.firstEmpty = ht.m$ then
@resize and rehash

$pos \leftarrow ht.h(k)$

if $ht.T[pos] = -1$ then //empty pos.
 $ht.T[pos] \leftarrow k$

$ht.next[pos] \leftarrow -1$

if $pos = ht.firstEmpty$ then
changeFirstE.

Search: 31

$h(31) = 31 \% 13 = 5 \rightarrow pos\ 5 \xrightarrow{next} pos\ 0 \xrightarrow{next} pos\ 1 \xrightarrow{next} pos\ 4$
⇒ found it.

39

$h(39) = 39 \% 13 = 0 \rightarrow pos\ 0 \xrightarrow{next} pos\ 1 \xrightarrow{next} pos\ 4 \xrightarrow{next} pos\ -1$
 $pos\ -1 \Rightarrow$ ~~39~~

Remove:

elems	3	11		8	
next	-1	-1	-1	0	-1

I.

remove(11)

mark it as empty

elems	3	////		8	
next	-1	-1	-1	0	-1

elems	20	56			
next	-1	-1	-1	-1	-1

II.

elems	20	11	56		
next	-1	2	-1	-1	-1

$$h(11) = 11 \% 5 = 1$$

$$h(20) = 20 \% 5 = 0$$

$$h(56) = 56 \% 5 = 1$$

remove(11)

$$h(56) = 56 \% 3 = 2$$

$$h(11) = 11 \% 3 = 2$$

$$h(20) = 20 \% 3 = 2 \rightarrow firstEmpty = 0 \Rightarrow put there$$

$$h(56) = 56 \% 3 = 2 \rightarrow firstEmpty =$$

But next of 11 is $\neq -1$

⇒ move 56 in place of 11; make next from 2 to be -1.

III. 1

elems	X	56	12	8	
next	-1	0	-1	-1	-1

= remove elem. from the end of a ll on array.

IV.

elems	X	56	12	1	
next	X	0	-1	-1	-1

V.

elems	X	56	X	13	
next	X	0	-1	-1	-1

-1

! Remove ex!